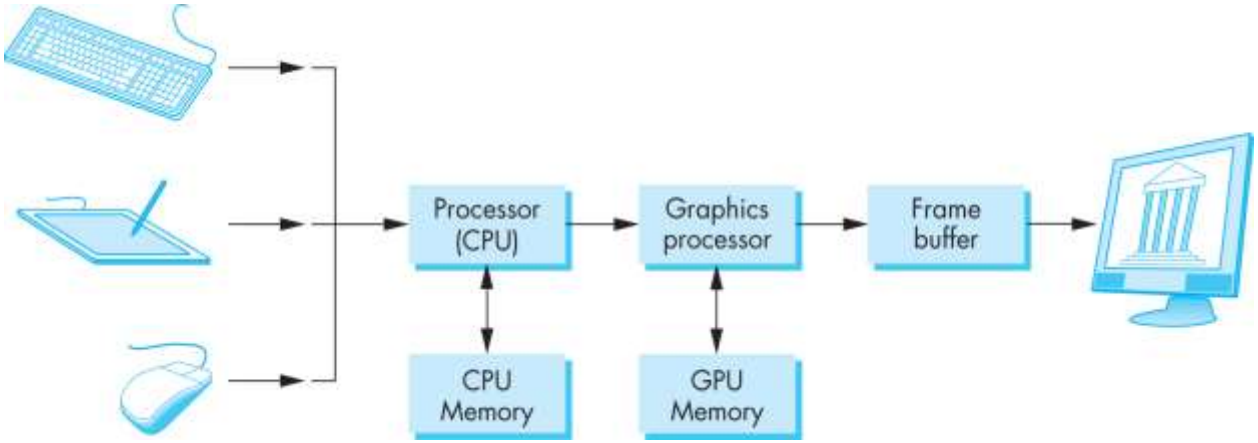
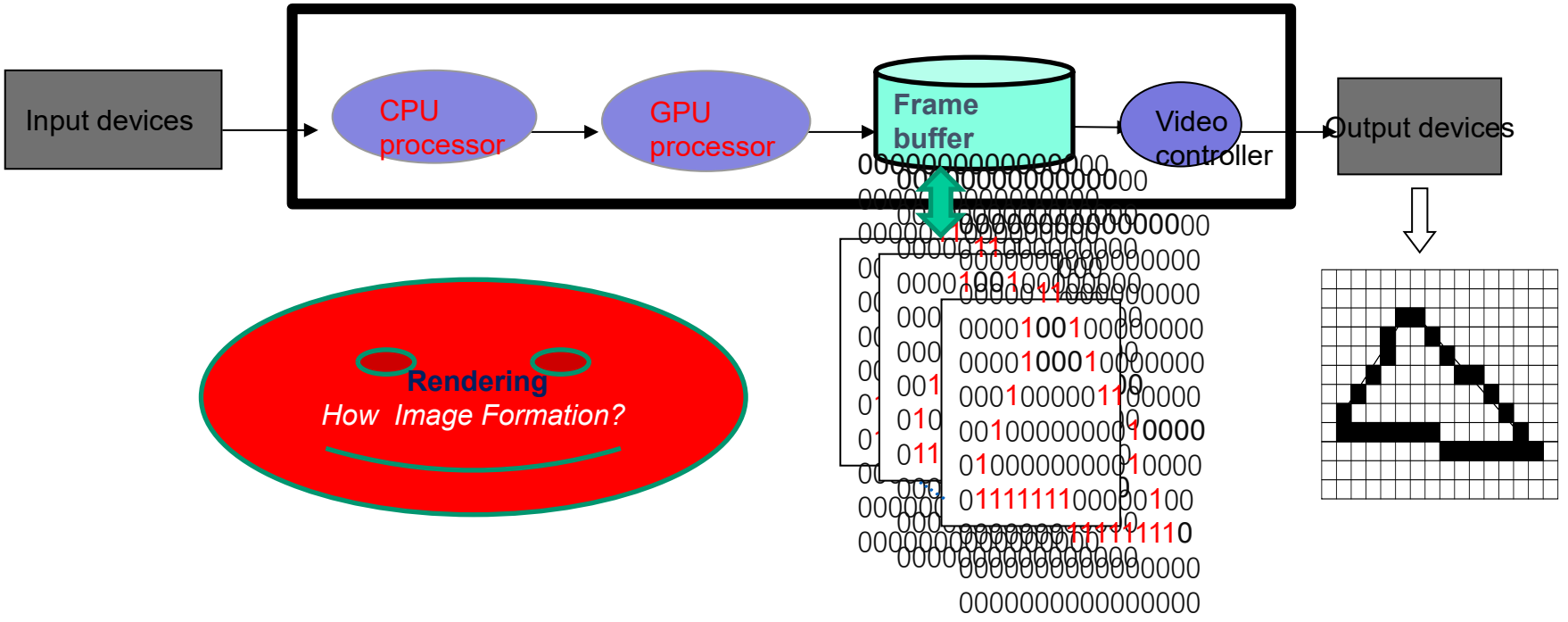
Recap: 图形系统

➢硬件



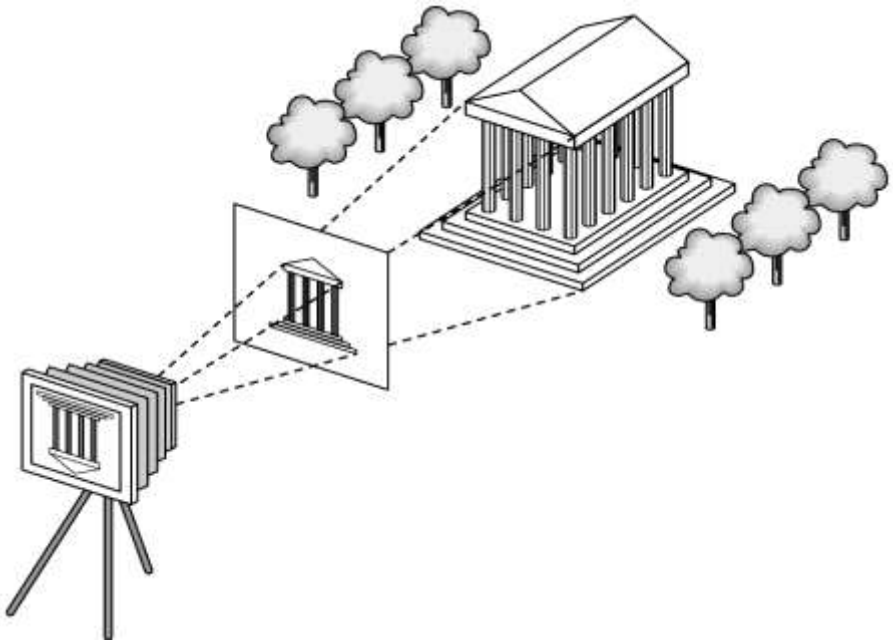


Recap: 图形系统(cont.)

◆成像模型：虚拟/合成相机模型

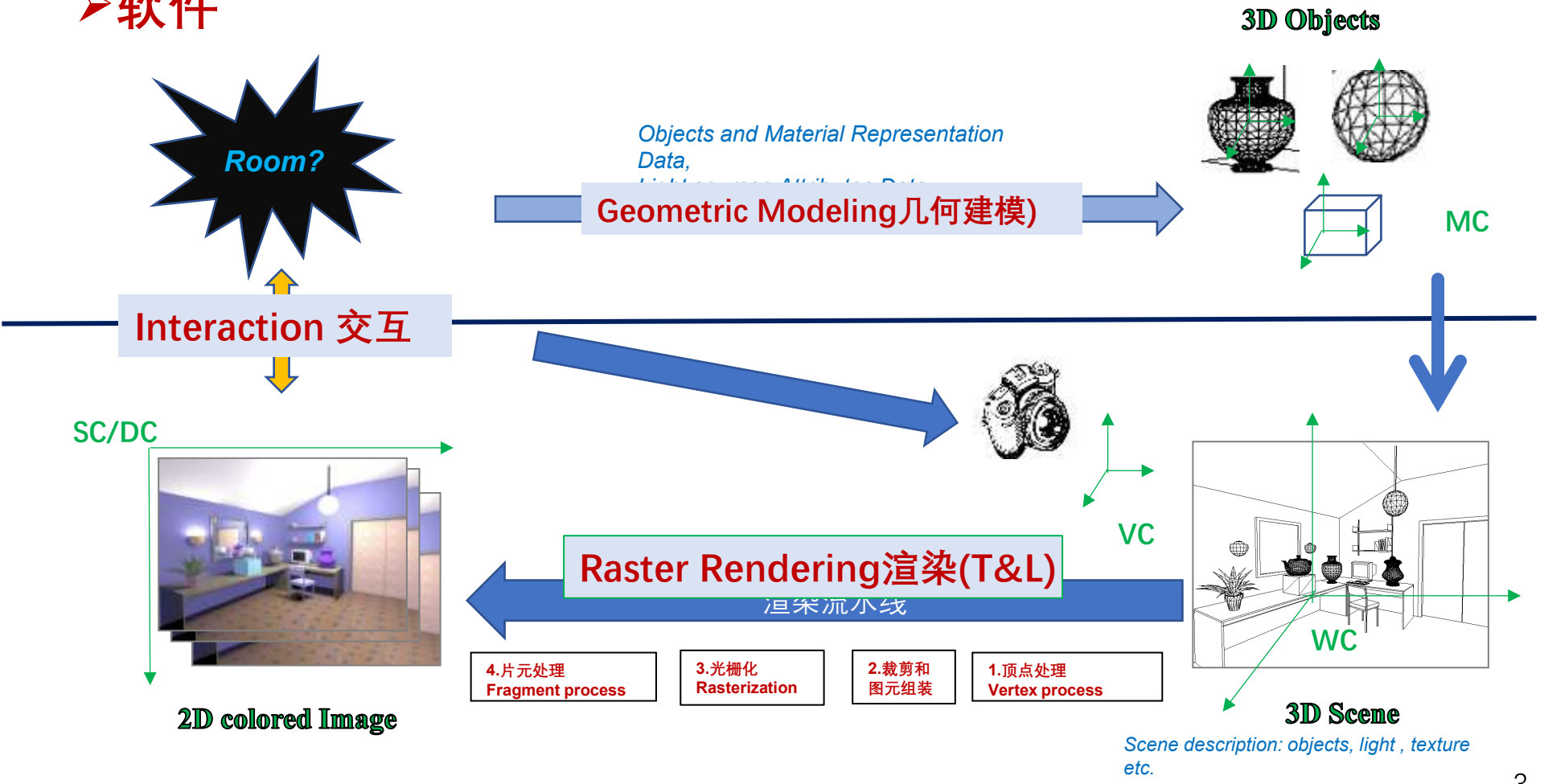
Three Elements of Image Formation

- 对象：Objects（物体对象）
  - 位置，材质
- 观察者：Viewer/Camera（观察者/相机）
  - 位置，相机朝向
- 光源：Light sources（光源）
  - 位置，颜色



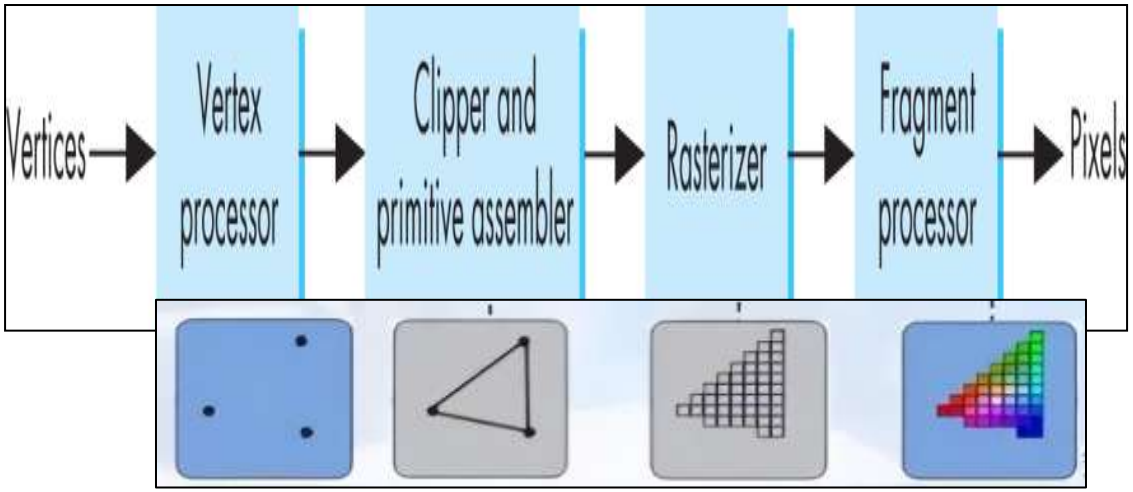
Recap: 图形系统(cont.)

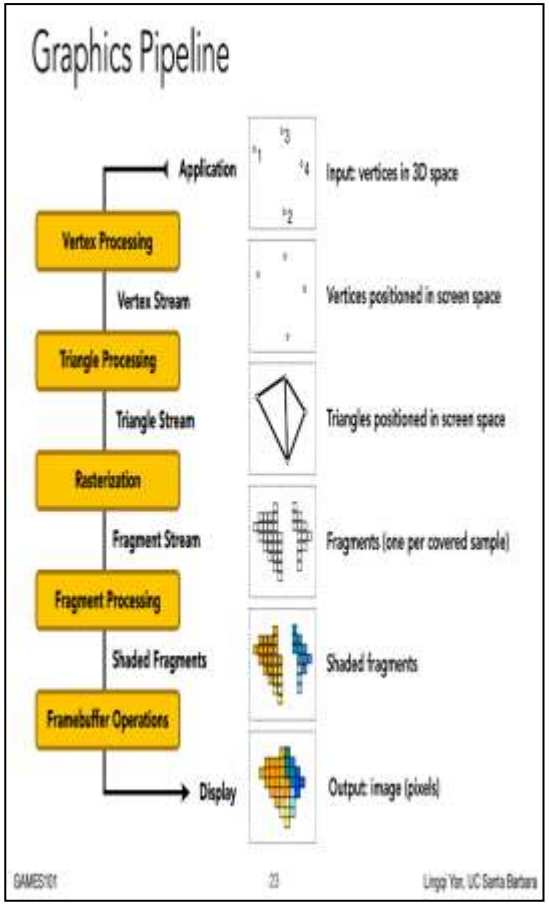
➢软件



Recap 图形系统(cont.)

- **管线实现Pipeline （基于网格表示的GPU绘制管线(T&L)的主要步骤）**
  1. **vertex process** 顶点处理
  2. **Clipper and Primitive assembling 裁剪和图元组装**
  3. **Rasterization 光栅化 (从图元到片元)**
  4. **fragment processing** 片元处理





Outlines

- Graphic Library Overview
- OpenGL Characteristic
- A Standard Program Structure using WebGL
- Shader Programing using GLSL
- Animation and Interaction using WebGL

Graphic Library Overview

- **史前时代：硬件专属的“黑暗年代”**
  - Graphical Kernel System (GKS)
    - 2D but contained good workstation model
    - GKS adopted as IS0 and later ANSI standard (1980s)
    - GKS not easily extended to 3D (GKS-3D), Far behind hardware development
  - Core ：Both 2D and 3D
  - Programmers Hierarchical Graphics System (PHIGS)
    - Arose from CAD community
    - Database model with retained graphics (structures)
  - X Window System
    - DEC/MIT effort
    - Client-server architecture with graphics
  - PEX combined the two PHIGS and X
    - Not easy to use (all the defects of each)

Graphic Library Overview (cont.)

➢开创先河:SGI的IRIS GL（1980年代）)

- Silicon Graphics (SGI)

- SGI（Silicon Graphics ）
- revolutionized the graphics workstation by implementing the pipeline in hardware (1982)

- GL

- application programmers can access the system
- With GL, it was relatively simple to program three dimensional interactive applications

Graphic Library Overview (cont.)

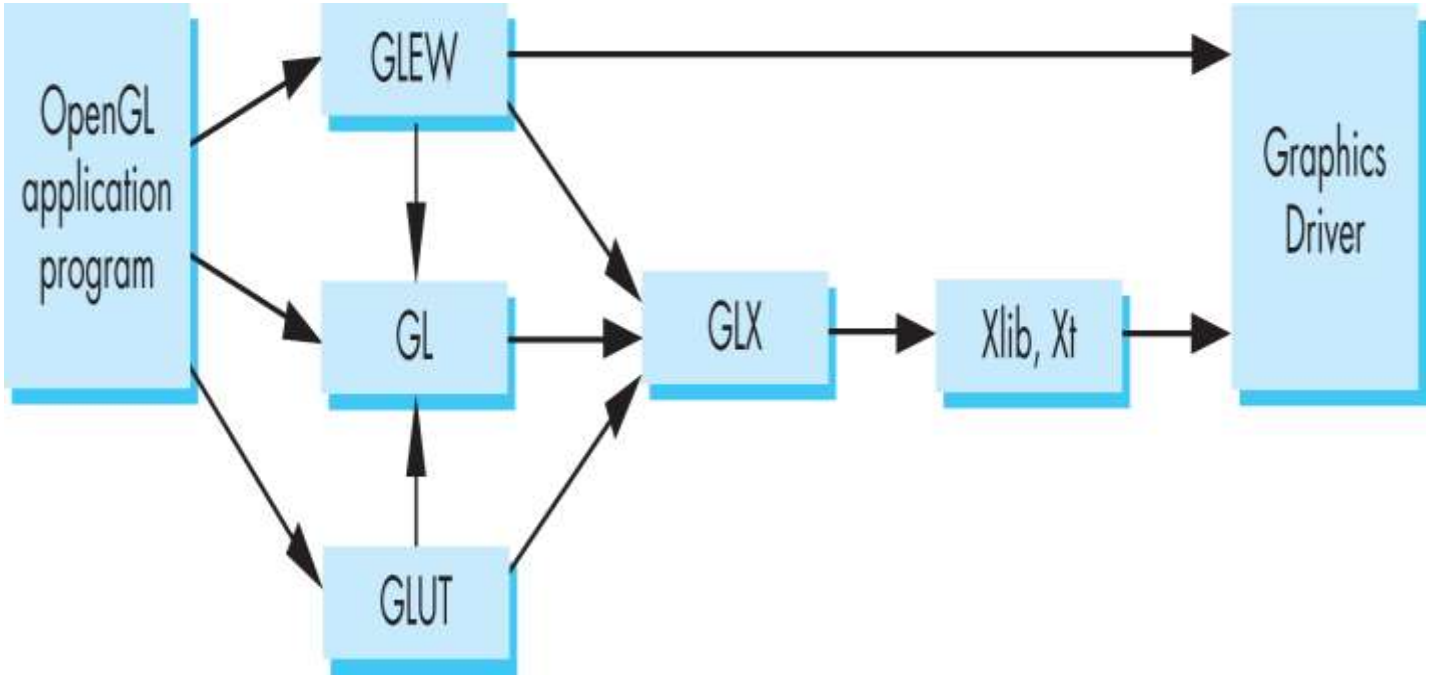
➢第一个真正的开放标准：OpenGL的诞生（1992)

- The success of GL lead to OpenGL (1992), "a platform-independent API ",
  - ***Easy to use***
  - ***Close enough to the hardware to get excellent performance***
  - ***Focus on rendering, Omitted windowing and input to avoid*** window system dependencies

- Originally controlled by an Architectural Review Board (ARB)(Members included SGI, Microsoft, Nvidia, HP, 3DLabs, IBM,…….), Now "Kronos Group"
  - **Relatively stable稳定 (through version 2.5)**
    - **Backward compatible**
    - **Evolution reflected new hardware capabilities**
      - 3D texture mapping and texture objects
      - Vertex and fragment programs……
  - **Allows platform specific features through extensions**

Graphic Library Overview (cont.)

- **openGL~Software Organization**
  - *GL:核心库,*
  - *GLEW:扩展,*
  - *GLUT:窗口管理*



Graphic Library Overview (cont.)

- Graphics Library Summary
  - **vulkan,DirectX,Metal,webGPU,这类API是当前发展的绝对主流，它们将取代传统的OpenGL/DX11，成为未来十年的中坚力量。**



未来的CG API图景是**分层和多元化的**:

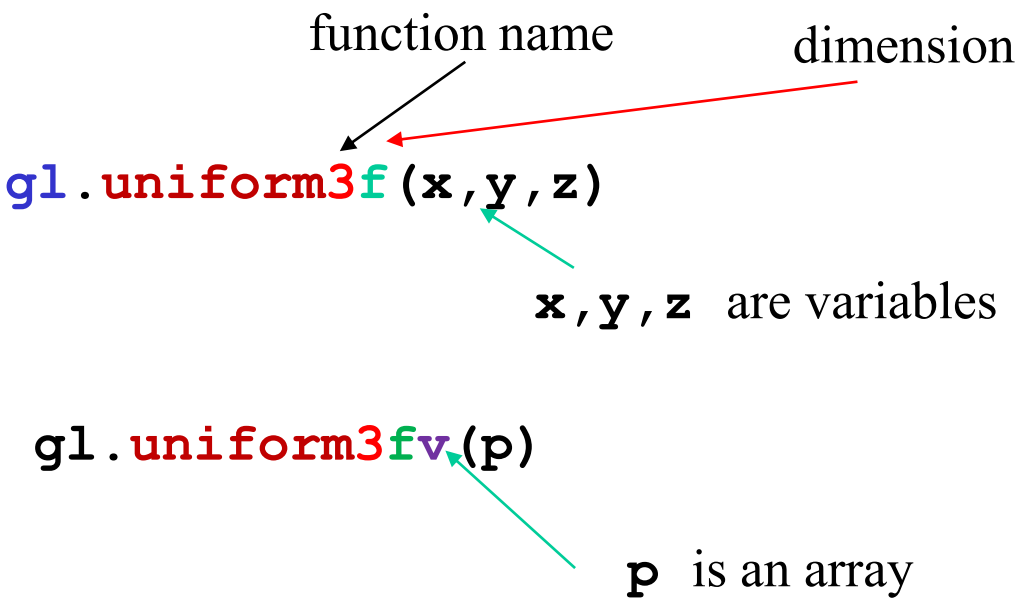
层次	代表	面向人群
底层硬件控制	**Vulkan, DX12, Metal**	引擎开发者、AAA游戏程序员、高性能计算专家
高级引擎与框架	**Unreal Engine, Unity**	游戏开发者、艺术家、应用开发者
Web与跨平台生态	**WebGPU**	Web开发者、跨平台应用开发者
特定领域技术	**DXR, OpenXR, DLSS SDK**	图形学研究员、VR/AR工程师、技术美术

Outlines

- Graphic Library Overview
- **GL Characteristic**
- **A Example using WebGL**
- **Shader Programing using GLSL**
- **Simple Interaction and Animation**

GL Characteristic~Formats

- **Function Formats**
  - Functions：gl 作为前缀
  - openGL Lack of Object Orientation: 缺少面向对象特性
    - All versions of OpenGL are not object oriented，so that there are multiple functions for a given logical function
      - **Example: Examples in WebGL:**



GL Characteristic~ Formats(cont.)

- **Constants Formats**
  - Constants（常量）: 大写，一般也有gl前缀
    - Most constants are defined in the canvas object
    - In desktop OpenGL, constands were in #include files such as `gl.h`

Example in OpenGL:

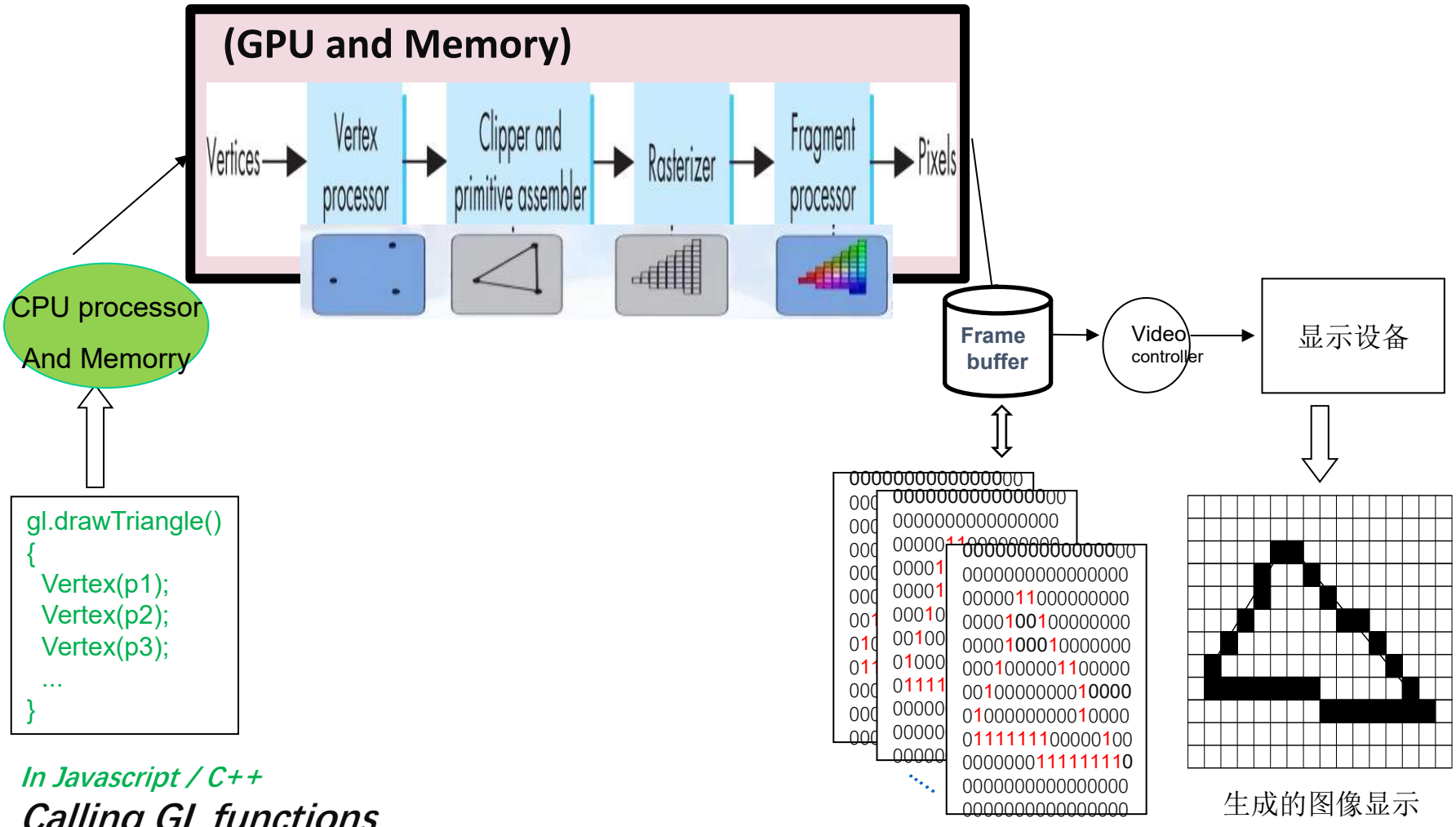
```
glEnable(GL_DEPTH_TEST);
glClear(gl.COLOR_BUFFER_BIT)
```

Examples in WebGL:

```
gl.enable(gl.DEPTH_TEST)
gl.clear(gl.COLOR_BUFFER_BIT)
```

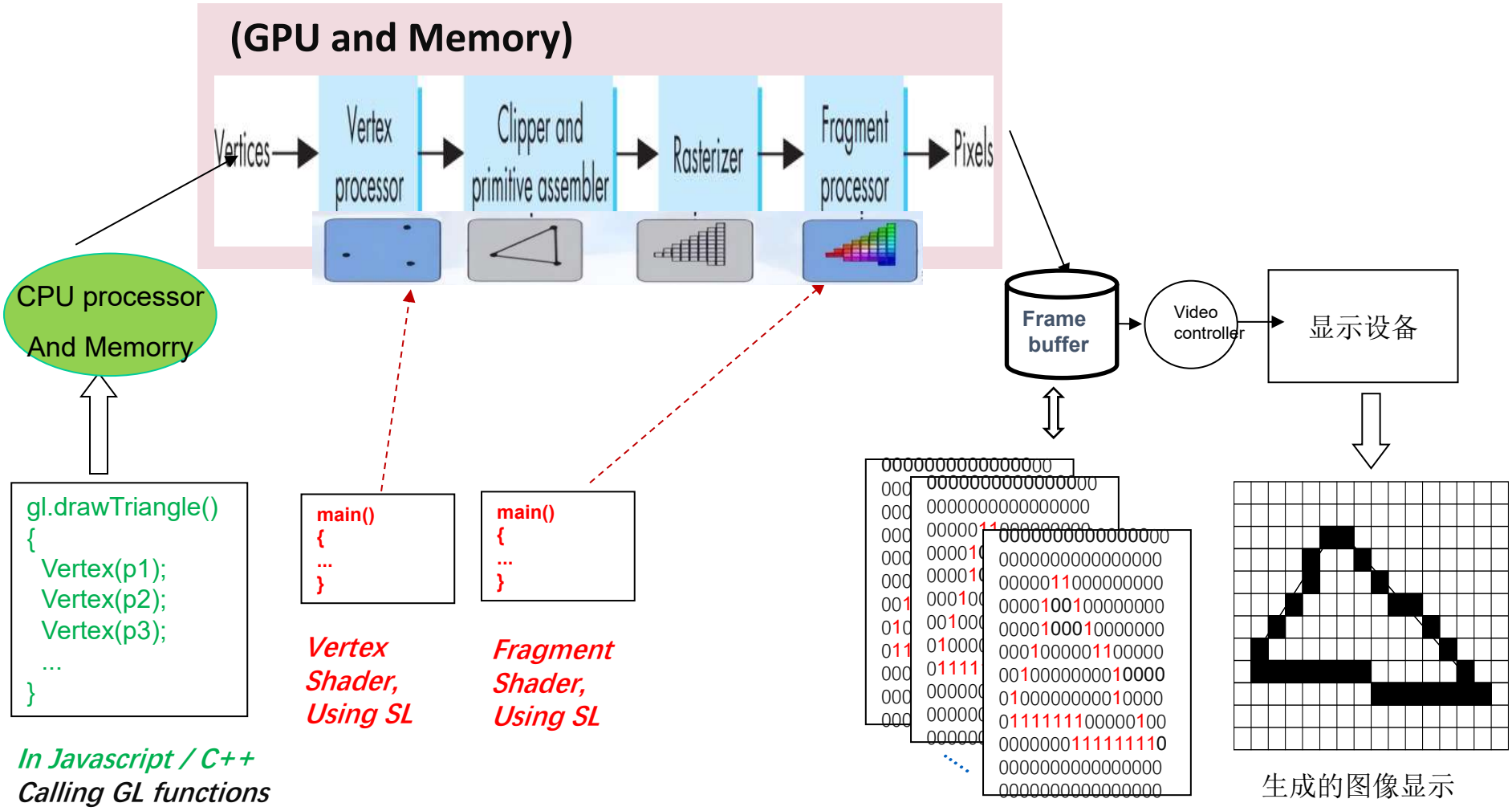
GL Characteristic~Architecture

➢ **The Fix Functions Rendering Pipeline : Black Box View!**



GL Characteristic~Architecture (cont.)

➢ **The Programmable Rendering Pipeline: Extend view!**



GL Characteristic~Architecture (cont.)

- **The Programmable Rendering Pipeline: Extend view!(cont.)**

  ◆ *Performance is achieved by using GPU rather than CPU!*

  1. **Application's job is to send data to GPU（传数据，配置参数等）**

  2. ***GPU does all rendering（并行处理）***
     - Performance is achieved by using GPU rather than CPU
     - Control GPU through programs called shaders（着色器程序）

  ◆ *后面以webGL为例子介绍*

Outlines

- Graphic Library Overview
- **GL Characteristic**
- **A Example Using WebGL**
- **Shader Programing using GLSL**
- **Simple Interaction and Animation**

A Example Using WebGL

- **openGL->WebGL**
  - Derivation：OpenGL, OpenGL ES, WebGL
  - **WebGL是一种JavaScript API，用于在任何兼容的网页浏览器中，不使用插件的情况下渲染2D图形和3D图形。**
  - 它基于OpenGL ES，是一个专为嵌入式系统设计的图形API。**兼容性**：WebGL 1.0基于OpenGL ES 2.0，WebGL 2.0基于OpenGL ES 3.0。**大多数现代浏览器都支持WebGL。**

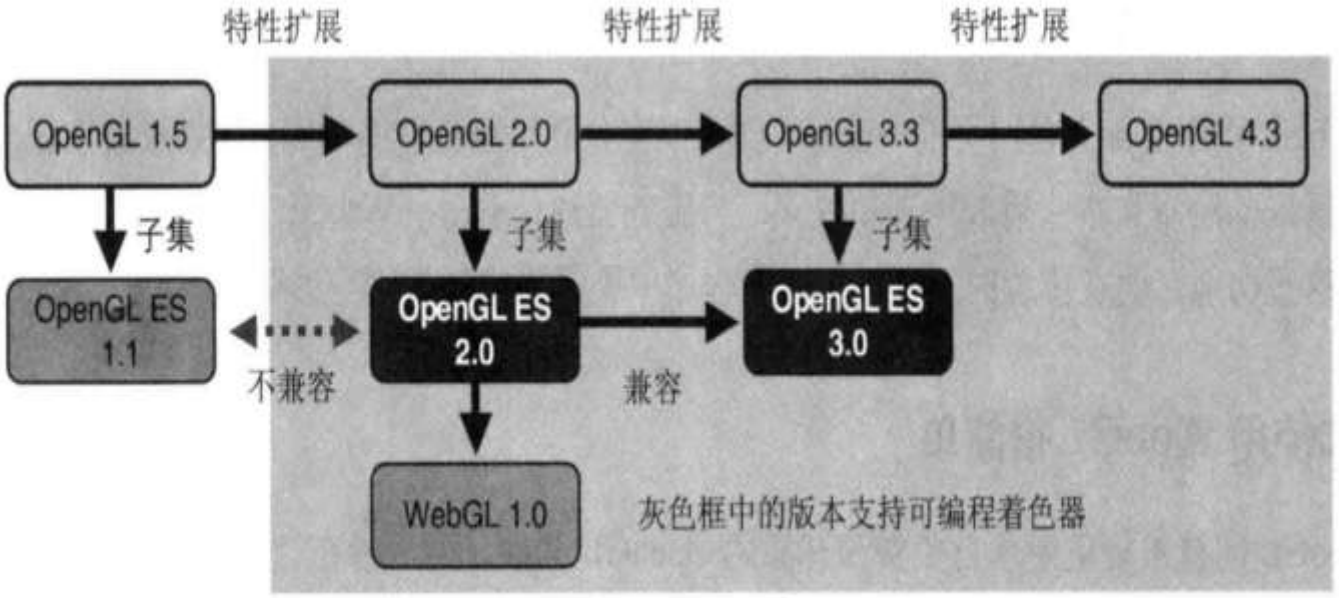


图 1.4 OpenGL、OpenGL ES 1.1//2.0/3.0 和 WebGL 之间的关系

A Example Using WebGL(cont.)

- 动态网页+3D图形绘制：Web(HTML5，JavaScript)+ WebGL(GL,GLSL)

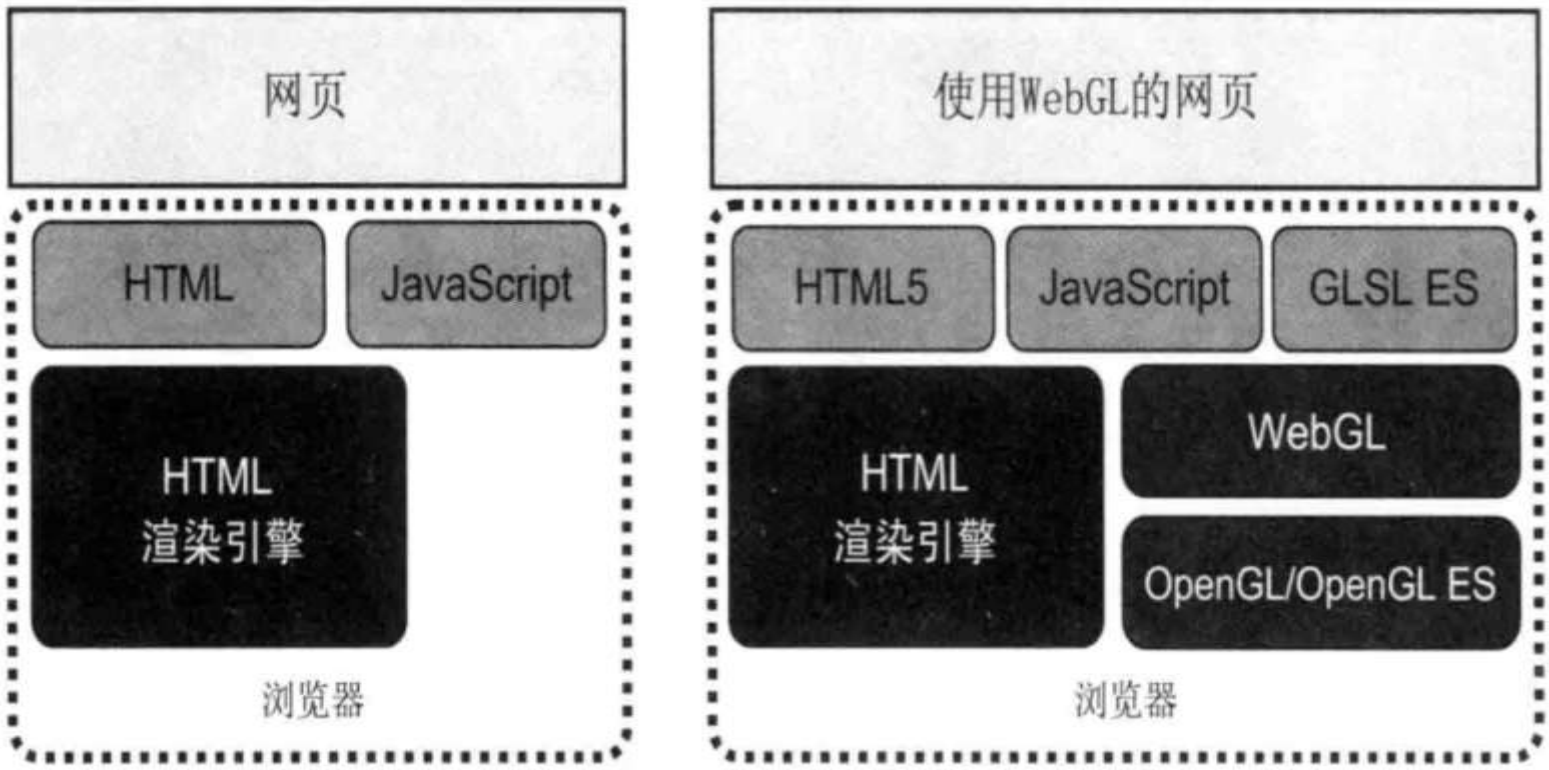
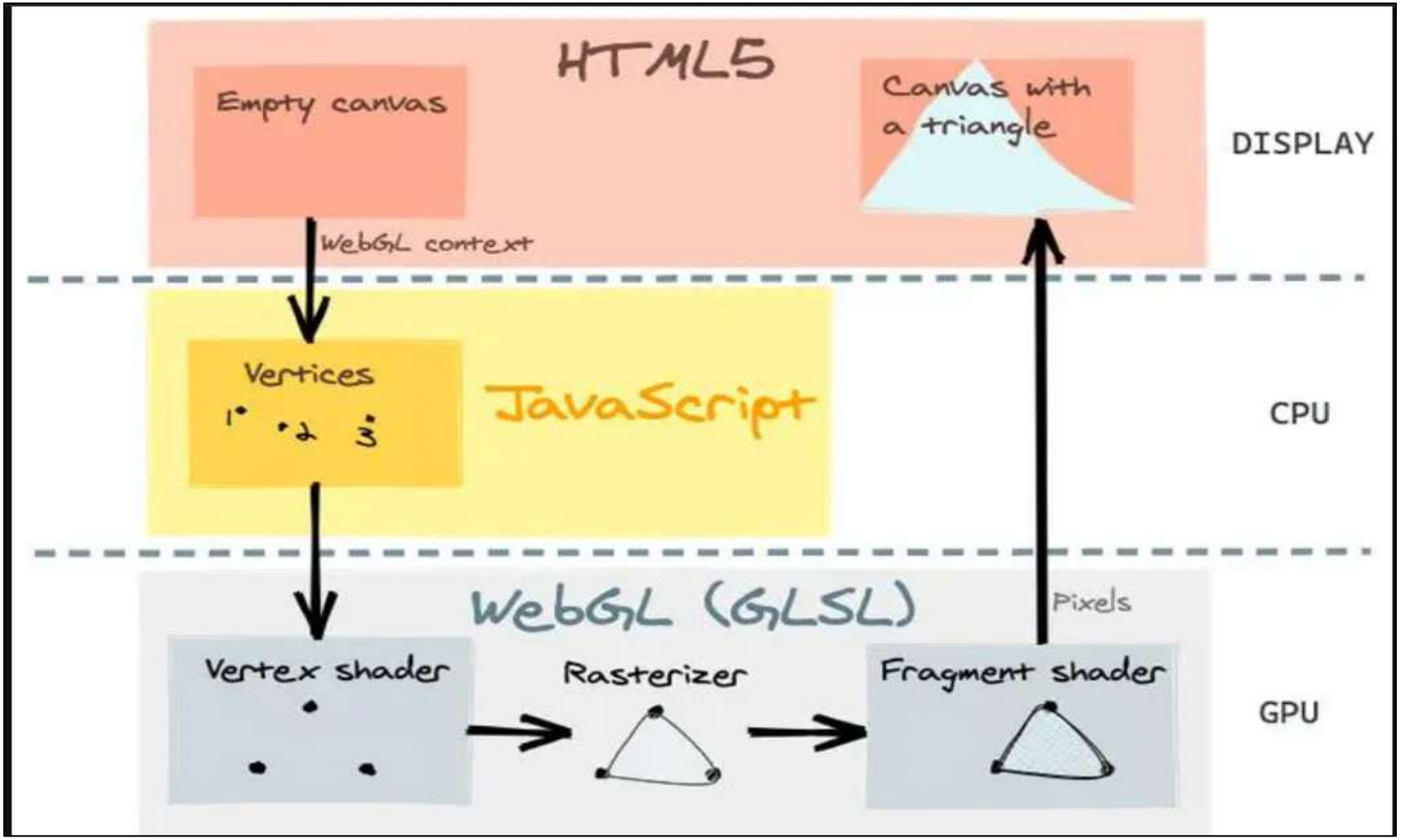


图 1.5 传统的动态网页（左侧）和 WebGL 网页（右侧）的软件结构

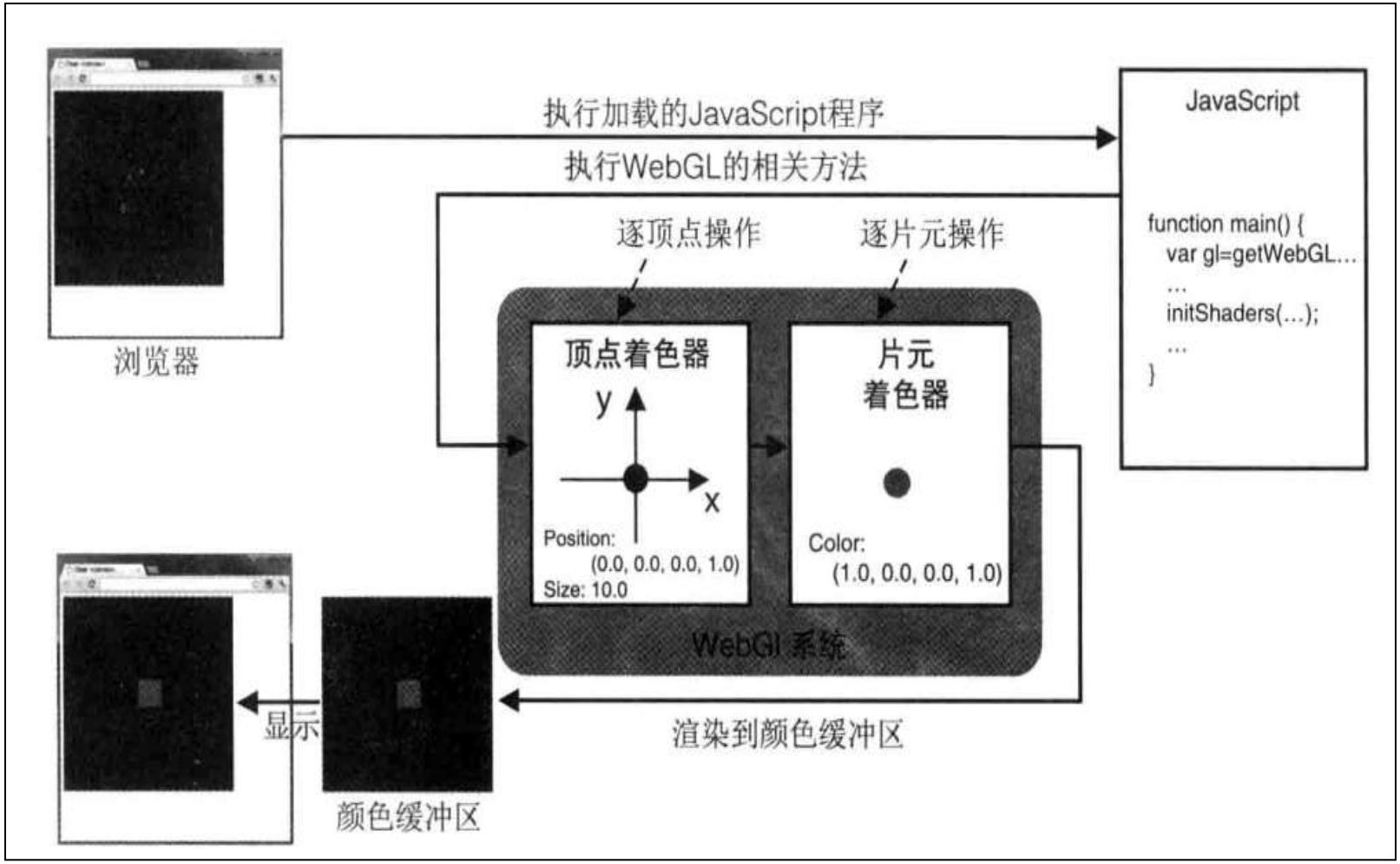
A Example Using WebGL（cont.)

- webGL程序架构



A Example Using WebGL（cont.)

- webGL程序执行过程



A Example Using WebGL（cont.)

- *Example：\Angle8E Code\02\square.**



- Program Five Parts
  1. Describe page (*HTML*)
  2. Config WebGL Environment(*JS*)
  3. Compute/Specify Geometry Data (*JS*)
  4. Send data to GPU and do Render (*JS*)
  5. Define Shaders (in any file with *GLSL*)
     1. could be done with a separate file (browser dependent)

A Example Using WebGL（cont.)

- **Part1: Describe page (*HTML*)** ***//square.html***
  - 定义界面内容：主要是定义“canvas”(画布)，以及界面的各种交互控件
  - 加载程序中需要的各个js文件<script type=…></script>

```
<!DOCTYPE html>
<html>
    <script id=“vertex-shader” type=“x-shader/x-vertex”> …… </script>          //本例专用-顶点着色器代码
    <script id=“fragment-shader” type=“x-shader/x-fragment”>…… </script> //本例专用-片元着色器代码
    <script type=“text/javascript” src=“../Common/initShaders.js”></script>   //例子通用-“着色器初始化”
    <script type=“text/javascript” src=“../Common/MV.js”></script>              //例子通用-“向量矩阵函数”
    <script type=“text/javascript” src=“square.js”></script>                      //本例专用-绘图代码
  <body>
    <canvas id="gl-canvas“  width="512"  height="512">
      Oops ... your browser doesn't support the HTML5 canvas element
    </canvas>
  </body>
</html>
```

A Example Using WebGL（cont.）

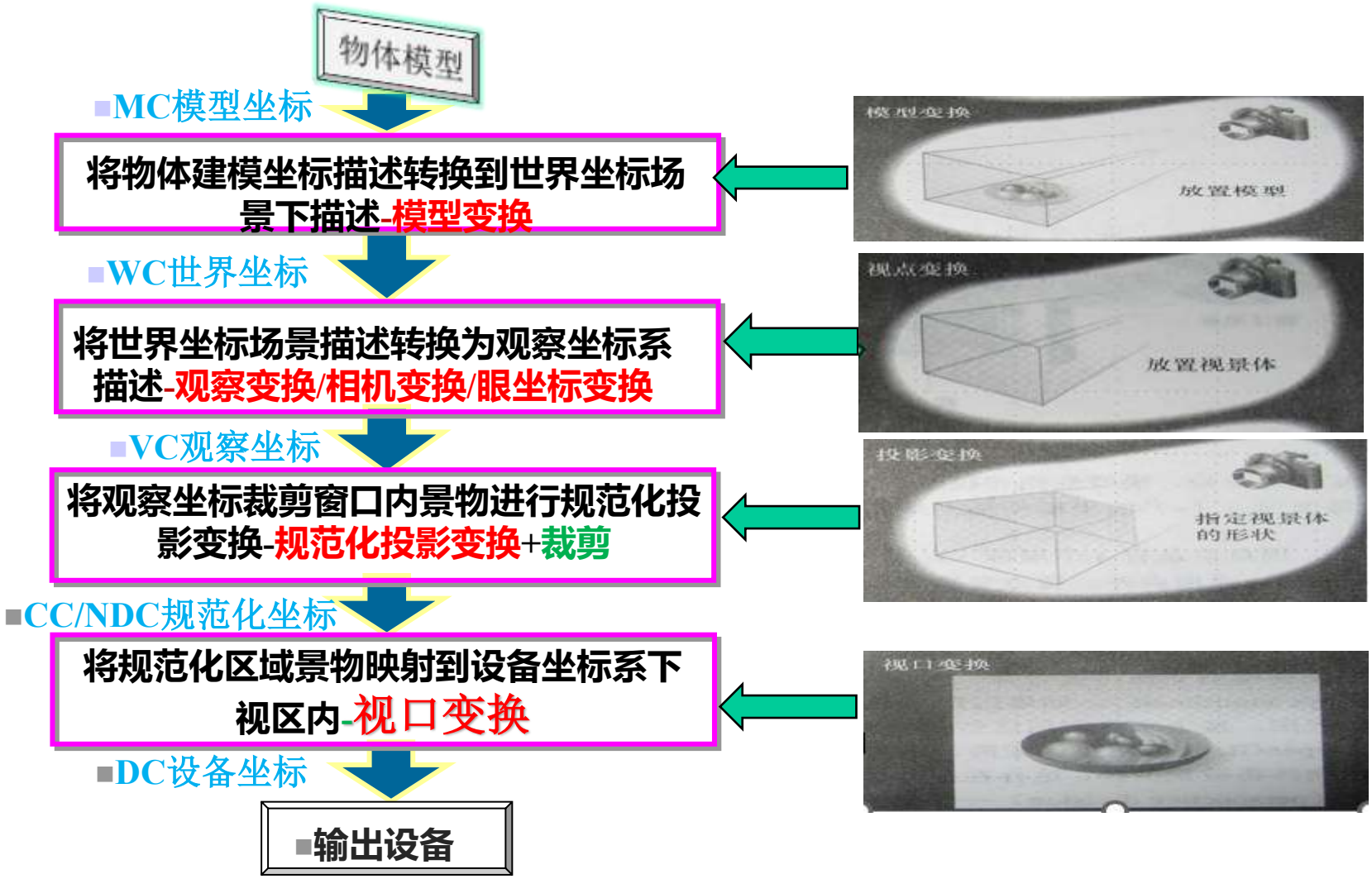
- **Part2：Config WebGL Environment(*JS*)** ***//square.js***
  - 创建绘图画布对象canvas：
    - 用于保存 HTML 中的 <canvas> 元素（绘图画布gl-canvas）
  - 创建绘图工具对象gl：
    - 用于保存 WebGL2.0渲染上下文对象，WebGL API 都是通过 gl.xxx() 来调用。
  - 配置webGL环境：
    - gl.viewport()：在画布上指定“绘图区域”（即：指出“视口”左下角像素位置（x，y）和“视口”的宽高（w,h））
    - gl.clearColor()：设置“清屏颜色”（即：指明 调用清屏命令gl.clear() 时，用来填充颜色缓冲区的**背景颜色**）

```
var canvas;
var gl;

window.onload = function init() {
    canvas = document.getElementById( "gl-canvas" );
    gl = canvas.getContext('webgl2');
    if ( !gl ) { alert( "WebGL 2.0 isn't available" ); }
    //Configure WebGL
    gl.viewport( 0, 0, canvas.width, canvas.height );
    gl.clearColor( 0.0, 0.0, 0.0, 1.0 );
```

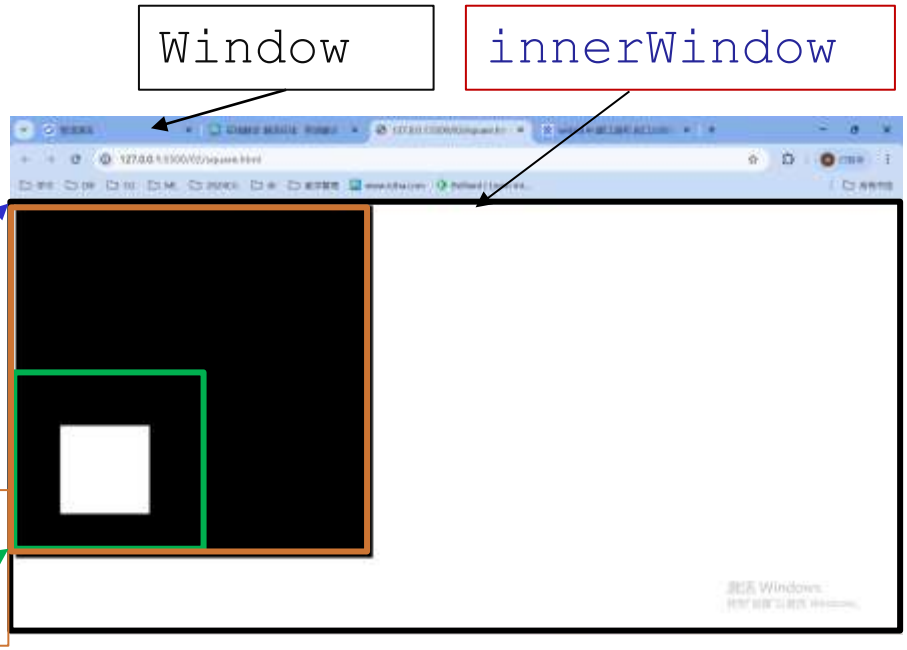
A Example Using WebGL(cont.)

➢从模型表示到设备输出:经历“五”种坐标系和“四”种变换！



A Example Using WebGL(cont.)

- Window, Canvas, Viewport
  - Viewport(视口)才是真正的webGL的“绘图区域”！



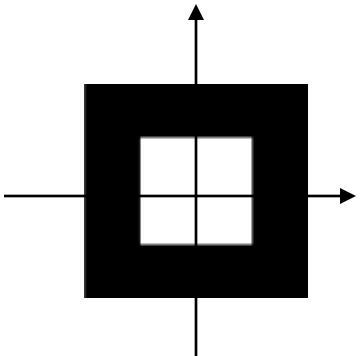

```
<canvas id="gl-canvas"
width="512" height="512">
//画布的坐标原点(0,0)在矩形左上角
```

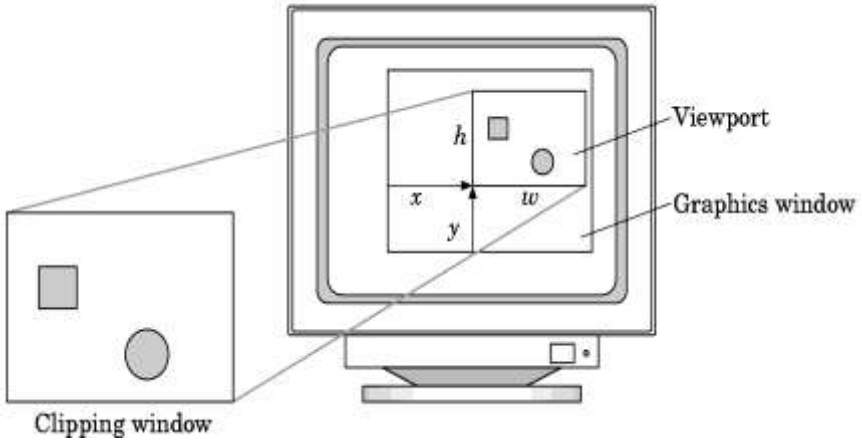
```
gl.Viewport(0, 0, w/2,h/2)
//在canvas中指定绘图区域:视口
//视口的坐标原点(0, 0)在矩形左下角！
```

A Example Using WebGL（cont.）

- **Part3. Compute/Specify Geometry Data (*JS*)**
  - 定义几何对象（指定图形顶点坐标）
    - “NDC坐标”：normalized device coordinates规范化坐标 (无量纲坐标)，有些也称为“裁剪坐标”，但是实际有一些区别。
      - 这里直接在“NDC坐标系”下表示几何图形的顶点位置
    - “裁剪窗口”： 取景矩形范围。在NDC坐标下一般规约到了[-1,-1][1,1]。
      - **注:“裁剪窗口clipping window ”与“视口viewport” 相对应！**

```
// Four Vertices of Square
    var vertices = [
        vec2( -0.5, -0.5 ),
        vec2(  -0.5,  0.5 ),
        vec2(  0.5, 0.5 ),
        vec2( 0.5, -0.5)  ];
```





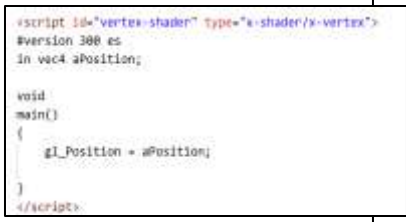
A Example Using WebGL（cont.）

- **Part4. Send data to GPU and do Render with Shader（*JS, shader*）**
  - 创建program对象，加载shader程序；
  - 创建‘顶点缓冲区对象VBO’**bufferID**， 并将’顶点位置数据’**vertices**放入
  - 创建‘缓冲区中的每顶点变量’**aPosition** **并关联** “顶点着色器中的每顶点变量”**aPosition**

```
var canvas;
var gl;
window.onload = function init() {
 ……
  //Load shaders and initialize attribute buffers
 var program = initShaders( gl, "vertex-shader", "fragment-shader" );
 gl.useProgram( program );

  // Load the data into the GPU
 var bufferId = gl.createBuffer();
 gl.bindBuffer( gl.ARRAY_BUFFER, bufferId );
 gl.bufferData( gl.ARRAY_BUFFER, flatten(vertices), gl.STATIC_DRAW );

  // Associate out shader variable with our data buffer
 var aPosition = gl.getAttribLocation( program, "aPosition" );
 gl.vertexAttribPointer( aPosition, 2, gl.FLOAT, false, 0, 0 );
 gl.enableVertexAttribArray(aPosition);
   ……
}  // flatten是例子公共包\Common\MV.js中定义的函数，用来“扁平化”顶点数组
```



A Example Using WebGL（cont.）

Part4. Send data to GPU and do Render with Shader

- 编写自己设计的渲染函数，一般取名render()
  - 首先，清屏gl.clear(); //用前面gl.clearcolor()设置的背景色清屏
  - 然后，用设置的图元绘制图形。 //用VBO中的顶点数据绘制图元

//注：图元 primitive：图形构成的基本单元，这里是三角扇

```
window.onload = function init() {
    ……
    render();
};

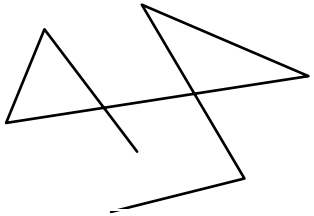
function render() {
    gl.clear( gl.COLOR_BUFFER_BIT );
    gl.drawArrays( gl.TRIANGLE_FAN, 0, 4 );//三角扇
}
```

A Example Using WebGL（cont.)

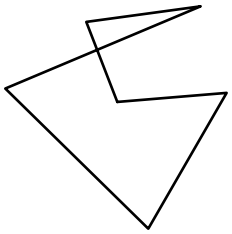
- WebGL Support Primitives（图元）



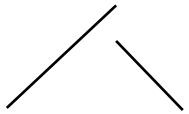
gl.POINTS



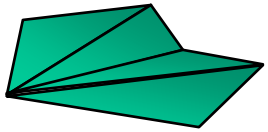
gl.LINE_STRIP



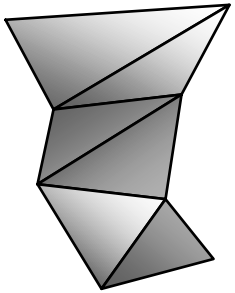
gl.LINE_LOOP



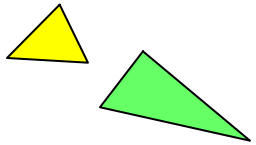
gl.LINES



gl.TRIANGLE_FAN



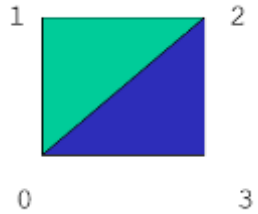
gl.TRIANGLE_STRIP



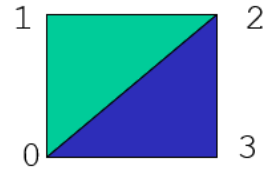
gl.TRIANGLES

A Example Using WebGL(cont.)

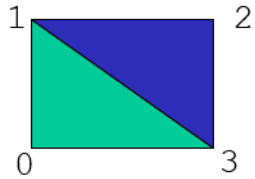
- webGL如何绘制一个四边形？
  - 可采用三种面图元(三角形，三角扇，三角带)绘制。
    - 注意:不同图元所需的顶点数，顶点组装顺序不同！



- gl.drawArrays(gl.TRIANGLES, 0, 6);
  - // 用三角形表示，
  - 需6个顶点数据，顺序可以为:0, 1, 2 , 2, 3, 0 (顺时针组装，如012,230)



- gl.drawArrays(gl.TRIANGLE_FAN, 0, 4);
  - 用三角扇表示，
  - 需用4个顶点数据，顺序为0, 1 , 2, 3 (组装为012,023)



- gl.drawArrays(gl.TRIANGLE_STRIP, 0, 4);
  - 用三角带表示，
  - 需用4个顶点数据，顺序为 0, 1, 3, 2 (组装为013,132)

A Example Using WebGL（cont.）

- Part5. Define Shaders (in any file with *GLSL*)
  - 着色器代码可在html文件中定义，也可在单独的文件中定义。
  - 每个图形应用至少定义一组着色器（含1个顶点着色器，1个片元着色器）

```
<html>
<script id="vertex-shader" type="x-shader/x-vertex">
#version 300 es
in vec4 vPosition;
void main()
{
    gl_Position = vPosition;
}
</script>

<script id="fragment-shader" type="x-shader/x-fragment">
#version 300 es
precision mediump float;
out vec4 fColor;
void main()
{
    fColor = vec4( 1.0, 1.0, 1.0, 1.0 );
}
</script>
```

Outlines

- GL History
- GL Characteristic
- A Example Using WebGL
- Shader Programing using GLSL
- Animation and Interaction using WebGL

Shader Programing using GLSL

- First programmable shaders were programmed in an assembly-like manner.
- OpenGL extensions added functions for vertex and fragment shaders
  - Cg (C for graphics) C-like language for programming shaders
    - Works with both OpenGL and DirectX
    - Interface to OpenGL complex
  - OpenGL Shading Language (GLSL)
    - Part of OpenGL 2.0 and up
    - High level C-like language
    - New data types:Matrices, Vectors, Samplers
  - As of OpenGL 3.1, application must provide shaders

Shader Programing using GLSL (cont.)

Data Types

- C types:
  - int, float, bool
- Vectors:
  - float vec2, vec3, vec4
  - Also int (ivec) and boolean (bvec)
- Matrices:
  - mat2, mat3, mat4
  - Stored by columns
  - Standard referencing m[row][column]
- C++ style constructors
  - vec3 a =vec3(1.0, 2.0, 3.0)
  - vec2 b = vec2(a)

Shader Programing using GLSL(cont.)

No Pointers

- There are no pointers in GLSL
- We can use C structs which can be copied back from functions
- Because matrices and vectors are basic types they can be passed into and output from GLSL functions, e.g.

  mat3 func(mat3 a)

- variables passed by copying

Shader Programing using GLSL (cont.)

Operators and Functions

- Standard C functions
  - Trigonometric
  - Arithmetic
  - Normalize(归一化), reflect, length
- Overloading of vector and matrix types

```
mat4 a;
vec4 b, c, d;
c = b*a; // a column vector stored as a 1d array
d = a*b; // a row vector stored as a 1d array
```

Shader Programing using GLSL（cont.)
Selection and Swizzling

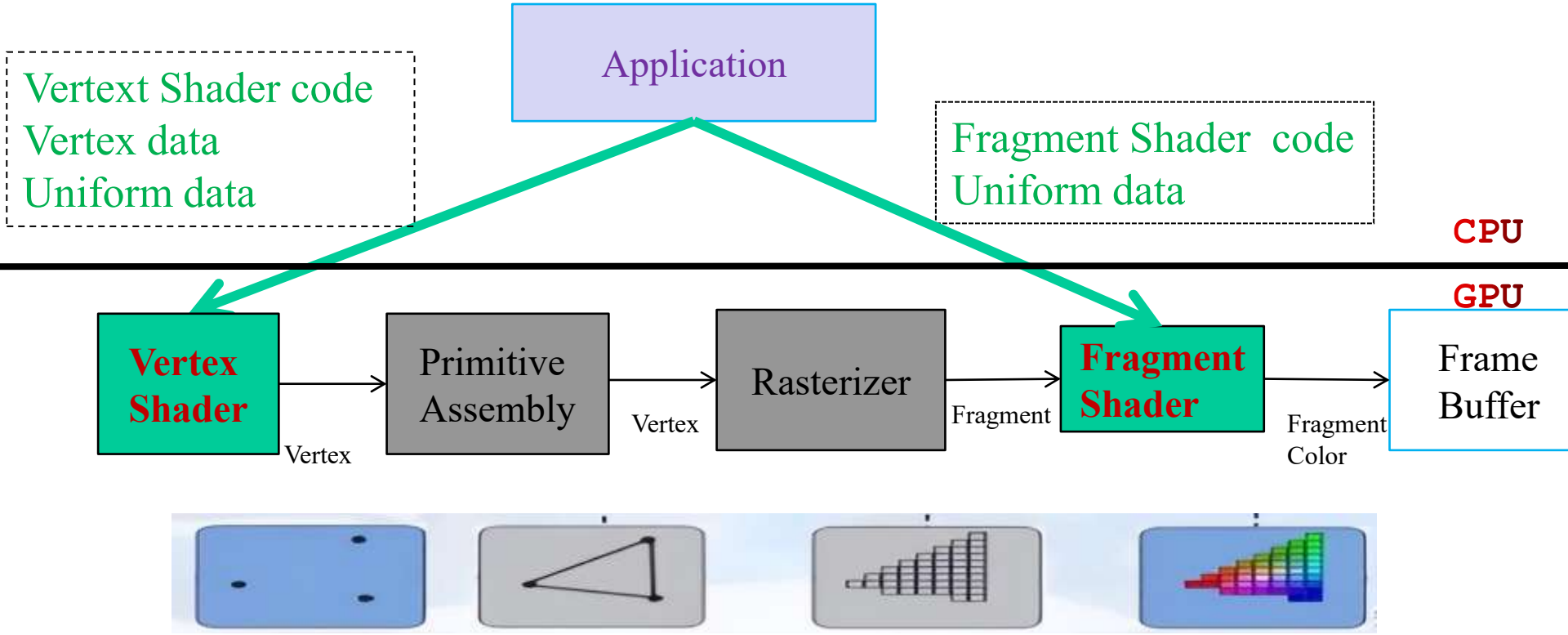
- Can refer to array elements by element using [] or selection (.) operator with
  - x, y, z, w
  - r, g, b, a:

  `Ex:a[2], a.b, a.z, a.p` are the same
- **Swizzling** operator lets us manipulate components

```
vec4 a, b;
a.yz = vec2(1.0, 2.0, 3.0, 4.0);
b = a.yxzw;
```

Shader Programing using GLSL（cont.）

- 基于三角网格表示的GPU渲染管线中有顶点处理和片元处理需要编写shader
  1. Vertex Shader：　对“每顶点”，进行顶点属性的操作
  2. Primitive Assembly：**对每三个顶点，组装为一个三角形图元**
  3. Rasterizer：**对每个三角形图元，将顶点属性 转换为 片元属性**
  4. Fragment Shader：　对“每片元”，进行片元颜色的操作



Shader Programing using GLSL（cont.)

- “Vertex Shader 顶点着色器”语法
  - webGL2.0中，必须标注版本version
  - webGL2.0中，用“in修饰符”替换了 “attribute修饰符 ”

Simple Vertex Shader (WebGL 2.0)

```
#version 300 es
in vec4 vPosition;
void main(void)
{
    gl_Position = vPosition;
}
```

compiler directive版本2.0标注

input from application

must link to variable in application

built in variable

Simple Vertex Shader (WebGL 1.0)

```
attribute vec4 vPosition;
void main(void)
{
    gl_Position = vPosition;
}
```

input from application

must link to variable in application

built in variable

Shader Programing using GLSL（cont.)

- “Fragment Shader 片元着色器”语法注意
  - webGL2.0中必须标注版本version，并且标注精度precision
  - webGL2.0中，输出片元颜色不再使用内置变量gl_FragColor

Simple Fragment Program (WebGL 2.0)

```
#version 300 es          <-- compiler directive
precision mediump float; <-- required
out fragColor;
void main(void)
{
   fragColor = vec4(1.0, 0.0, 0.0, 1.0);
}
```

Simple Fragment Program (WebGL 1.0)

```
precision mediump float;
void main(void)
{
  gl_FragColor = vec4(1.0, 0.0, 0.0, 1.0);
}
```

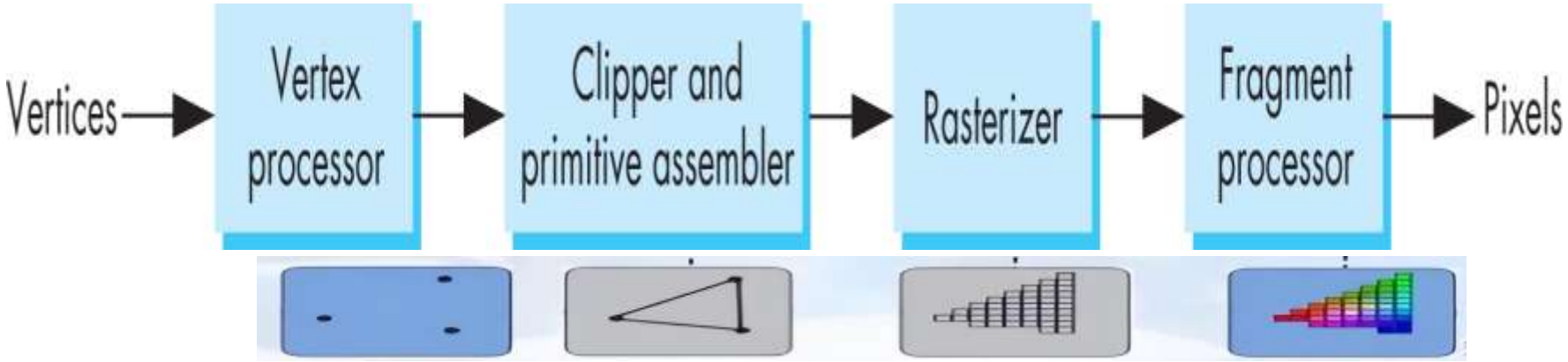
built in variable

Shader Programing Using GLSL（cont.)

- Variable Qualifiers变量修饰符说明：
  - **webGL1.0中用“Attribute”， webGL2.0中调整为“In”**
    - Attribute-qualified variables can change at most once per vertex
    - 一般只修饰顶点着色器中，输入的顶点属性如顶点位置，法向量，纹理坐标，顶点颜色等
  - **webGL1.0中的“Varying”，webGL2.0中调整为“Out”和“In”**
    - Variables that are passed from vertex shader to fragment shader
      - **out vec4 color; // at vertex shader，要求变量“同名”**
      - **in vec4 color; // at fragment shader, 要求变量“同名”**
  - **Uniform（统一的，一致的）**
    - Variables that are constant for an entire primitive(对整个图元来说，该变量是一个常量）
    - Used to pass information to shader such as the time or a bounding box of a primitive or transformation matrices,（用于向着色器传递统一的信息（例如时间、图元的边界框或变换矩阵等））
    - Can be changed in application and sent to shaders，Cannot be changed in shader（在应用程序中可修改其值并发送给着色器,但不能在着色器中修改）

Shader Programing using GLSL（cont.）

- Variable Qualifiers变量修饰符（cont.）
  - attribute（in), varying(in/out) , uniform




```
//Attribute:每顶点不同的属性
In vec4 aPosition;
In vec4 aColor
```

```
//Varying:  每顶点传出值，后续将进行光栅化插值计算
out vec2 fColor;
```

图元组装，光栅化插值

```
//Varying:每片元传入值，经过"光栅化插值"后得到的每片元的属性值
in vec2 fColor;
```

```
//uniform:每顶点一致的变量
Uniform mat4 uModelMatix;
```

```
//uniform 每片元一致的变量
Uniform vec4 lightColor;
```

Shader Programing using GLSL（cont.)

- Variable Qualifiers变量修饰符（cont.)

Example1：在片元着色器里设置颜色 Sending a Uniform Variable to Set Fragment Color

```
// in application
vec4 aColor = vec4(1.0, 0.0, 0.0, 1.0);
gl.uniform4f(gl.getUniformLocation( program, ”color" ), aColor);
```

```
// in fragment shader (similar in vertex shader)
uniform vec4 aColor;
out vec4 fColor;
void main()
{
    fColor = aColor;
}
```

Shader Programing using GLSL（cont.）

- Variable Qualifiers变量修饰符（cont.）

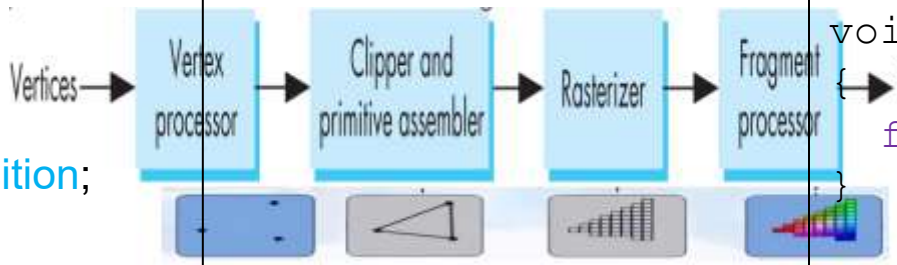
Example2: 在应用程序里设置顶点颜色Set Colors in Application

先传给顶点着色器，然后直接out输出给后面的模块（图元组装和光栅化）

经过光栅化后，顶点色插值后生成每片元的颜色，每片元颜色作为in变量输入每个片元着色器

```
var cBuffer = gl.createBuffer();
gl.bindBuffer( gl.ARRAY_BUFFER, cBuffer );
gl.bufferData( gl.ARRAY_BUFFER, flatten(colors),  gl.STATIC_DRAW );
var aColor = gl.getAttribLocation( program, ”aColor" );
gl.vertexAttribPointer( aColor, 3, gl.FLOAT, false, 0, 0 );
gl.enableVertexAttribArray( aColor );
```

```
/*** vertex shader****/
in vec4 aColor, aPosition;
out vec4 vColor ;
void main()
{
  gl_Position = aPosition;
   vColor = aColor;
}
```




```
/***fragment shader****/
precision mediump float;
in vec4 vColor;
out vec4 fColor;
void main()
{
  fColor = vColor;
}
```

Summary

- GL History
- GL Characteristic
- A Example Using WebGL
- Shader Programing using GLSL
- **Animation and Interaction using WebGL**

Animation Type and FPS

- **Animation:** *Reference: GAMES101_Lecture21*

"Bring things to life"

- Communication tool
- Aesthetic issues often dominate technical issues

An extension of modeling

- Represent scene models as a function of time

Output: sequence of images that when viewed sequentially provide a sense of motion



(Phenakistoscope, 1831)

Animation Type and FPS(cont.)

Animation Types

- 逐帧动画（frame-by-frame animation）
  - Offline generation and real-time playback.
  - Such as: movies, TV
- 实时动画（Online real-time animation）：
  - Online generated and play immediately.
  - Such as: games, interactive applications.



Animation Type and FPS(cont.)

- Frame frequency(帧频)：**frames per second (FPS)**
  - 帧率能够达到 50 ～ 60 FPS 的动画将会相当流畅，让人倍感舒适；
  - 帧率在 30 ～ 50 FPS 之间的动画，因各人敏感程度不同，舒适度因人而异；
  - 帧率在 30 FPS 以下的动画，让人感觉到明显的卡顿和不适感；
  - 帧率波动很大的动画，亦会使人感觉到卡顿。
- Display Refresh frequency:显示器刷新频率
  - 每秒刷新60次以上，**屏幕显示感觉不到闪烁**

Output: sequence of images that when viewed sequentially provide a sense of motion

- Film: 24 frames per second
- Video (in general): 30 fps
- Virtual reality: 90 fps

Animation Type and FPS(cont.)

- FPS>150：帧拖延效果（晕炫视觉，拖影）
  - 原因~ 若创建一帧时间很短，远远小于屏幕的刷新周期，则导致拖影。
  - ✓**solution：Time Delay时延**

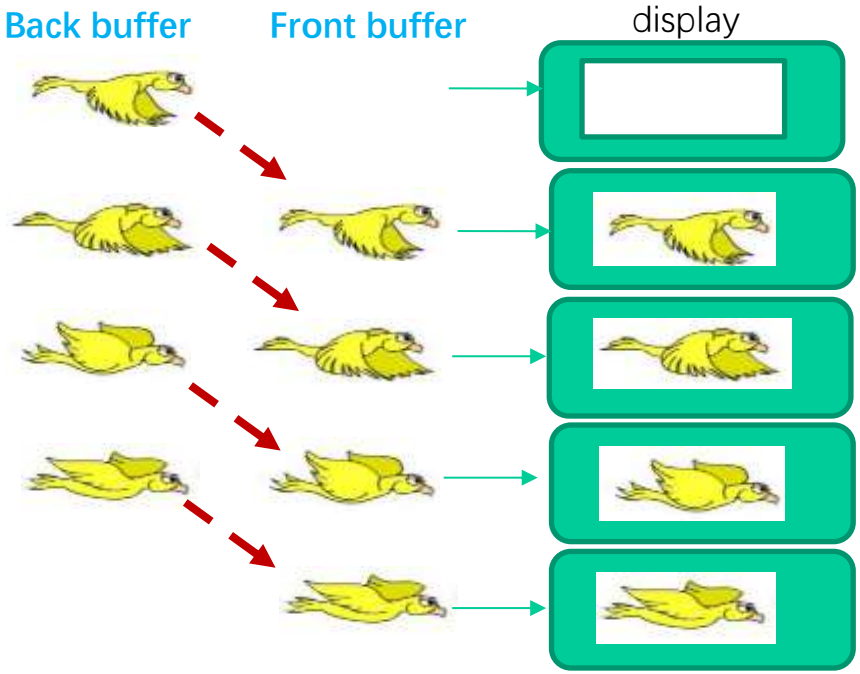
- FPS<24：帧断裂 or 帧飘移 (破帧)
  - 原因~ 单帧缓存(只有1个颜色帧缓存)：若创建一帧时间大于 屏幕一个刷新周期，则会partial rendering:部分绘制，导致帧飘移或帧破裂。
  - ✓**Solution：Double Buffering双帧缓存**

Double Buffering for Smooth Animations



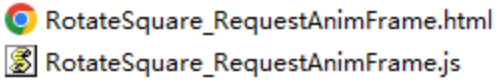
- Double Buffering
  - when we are rendering a frame, it always be render into a buffer that is not displayed to prevents display of a “partial rendering”.
  - front buffer前帧: always display
  - back buffer后帧: rendering into

openGL use a buffer swap



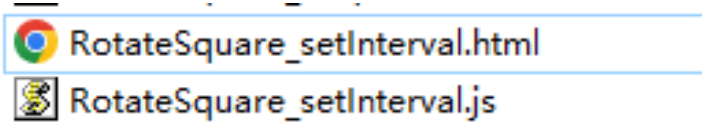
Animation in Brower

- requestAnimationFrame()
  - The function is now part of JS ,
  - Browser refresh the display at 60 Hz, this function will allow only one execution per display refresh




```
function render {
    gl.clear(gl.COLOR_BUFFER_BIT);
    uTheta += 0.1;
    gl.uniform1f(thetaLoc, uTtheta);
    gl.drawArrays(gl.TRIANGLE_STRIP, 0, 4);
    requestAnimationFrame(render);
}
```

Animation in Brower(cont.)

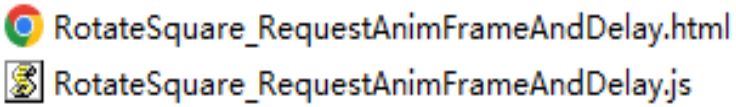


- Interval Timer()
  - Executes a function after a specified number of milliseconds
  - setInterval(render, interval);

- Note an interval of 0 generates buffer swaps as fast as possible,
- Defects:
  - Stack overflow
  - 每种浏览器中可能有区别，难以得到平滑动画显示！不兼容

Animation in Brower(cont.)

- Combination of “requestAnimationFrame” and “Interval timer”



➢setTimeout()

```
function render()
{
    gl.clear(gl.COLOR_BUFFER_BIT);

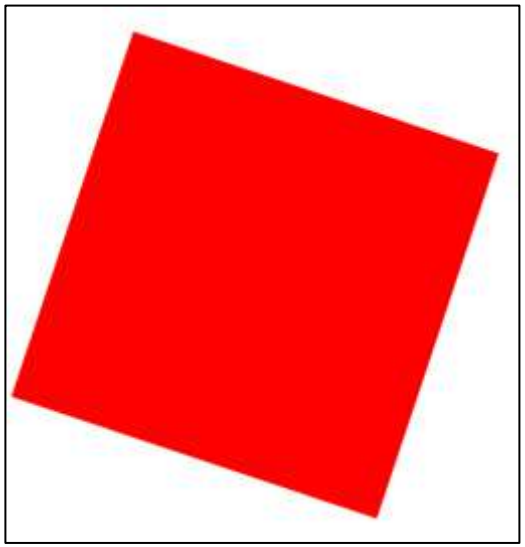
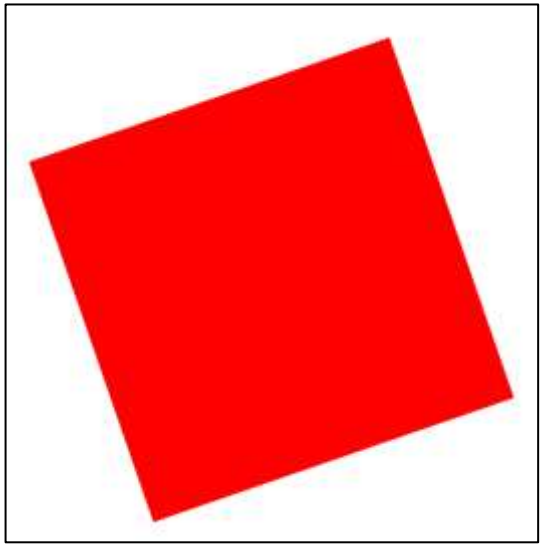
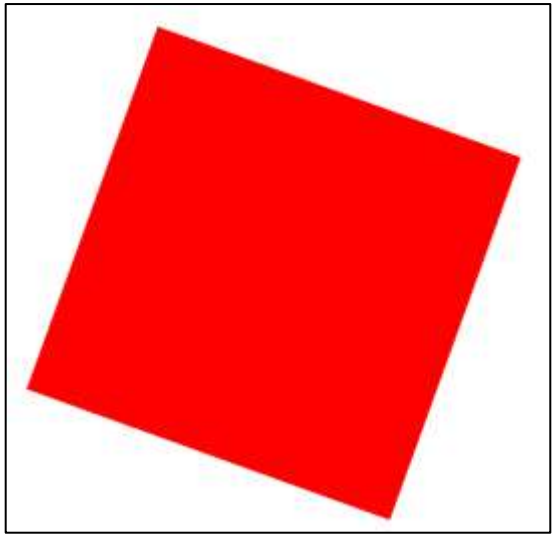
    theta += (direction ? 0.1 : -0.1);
    gl.uniform1f(thetaLoc, theta);

    gl.drawArrays(gl.TRIANGLE_STRIP, 0, 4);

    setTimeout(
        function (){requestAnimationFrame(render);}, delay
    );
}
```

Animation Example

- Rotated Square
  - Animate display by rerendering with different values of θ

Animation Example(cont.)

- 绘制方法1:在主程序中计算下一帧图形顶点后再绘制
  - Recompute the vertices locations by new theta value
  - Resend new data array from CPU to GPU
  - ✓Simple but slow! //see RotateSquare_Interaction

```
for(var theta = 0.0; theta <thetaMax; theta += dtheta; ) {
    vertices[0] = vec2(Math.sin(theta), Math.cos.(theta));
    vertices[1] = vec2(Math.sin(theta), -Math.cos.(theta));
    vertices[2] = vec2(-Math.sin(theta), -Math.cos.(theta));
    vertices[3] = vec2(-Math.sin(theta), Math.cos.(theta));

    gl.bufferData(…)

     render();
}
```

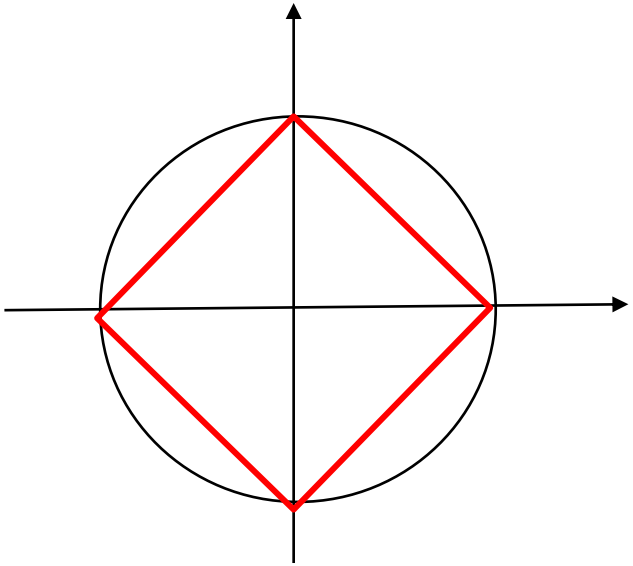
Animation Example(cont.)

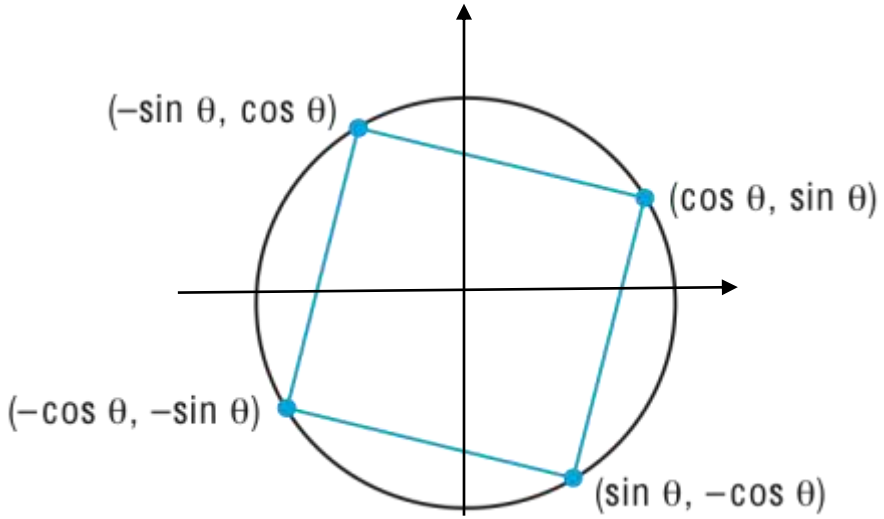
上页中新帧中正方形顶点计算方法如下：

//注意：四个顶点的计算方法各自不同

$$x_{i+1} = R\cos\theta_{i+1}$$

$$y_{i+1} = R\sin\theta_{i+1}$$





Consider the four point, circle radius=1

Animation Example(cont.)

绘制方法2:在“顶点着色器里”重新计算下帧顶点位置并绘制,

- Send original vertices to vertex shader(only once)
- Send new θ to shader as a uniform variable, and **Compute new vertices in vertex shader**
- Better Way, *//see “rotatingSquare1*

```
var vertices = [
    vec2(0, 1),
    vec2(-1, 0),
    vec2(1, 0),
    vec2(0, -1)
];

// Load the data into the GPU
var bufferId = gl.createBuffer();
gl.bindBuffer(gl.ARRAY_BUFFER, bufferId);
gl.bufferData(gl.ARRAY_BUFFER, flatten(vertices), gl.STATIC_DRAW);

// Associate out shader variables with our data bufferData
var positionLoc = gl.getAttribLocation(program, "aPosition");
gl.vertexAttribPointer(positionLoc, 2, gl.FLOAT, false, 0, 0);
gl.enableVertexAttribArray(positionLoc);

thetaLoc = gl.getUniformLocation(program, "uTheta");

render();
};
```

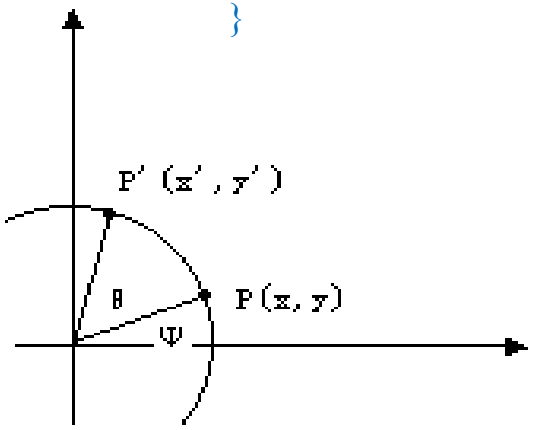
```
function render()
{    gl.clear(gl.COLOR_BUFFER_BIT);
    uTheta += 0.1;
    gl.uniform1f(thetaLoc, uTheta);
    gl.drawArrays(gl.TRIANGLE_STRIP, 0, 4);
    render();
}
```

```
in vec4 aPosition;
uniform float uTheta;
void main()
{
  gl_Position.x = -sin(uTheta) * aPosition.x + cos(uTheta) * aPosition.y;
  gl_Position.y = sin(uTheta) * aPosition.y + cos(uTheta) * aPosition.x;
  gl_Position.z = 0.0;
  gl_Position.w = 1.0;
}
```

Animation Example(cont.)

- 上页中，顶点着色器中的每顶点的计算公式相同！
- P是初始顶点位置，P’是旋转theta后的顶点位置，可以推出它们之间的计算关系

```
in vec4 aPosition;
uniform float uTheta;
void main()
{
  gl_Position.x = -sin(uTheta) * aPosition.x + cos(uTheta) * aPosition.y;
  gl_Position.y = sin(uTheta) * aPosition.y + cos(uTheta) * aPosition.x;
  gl_Position.z = 0.0;
  gl_Position.w = 1.0;
}
```



$x' = r\cos(\theta+\Psi)$

$= r\cos\theta\cos\Psi - r\sin\theta\sin\Psi$

$y' = r\sin(\theta+\Psi)$

$= r\sin\theta\cos\Psi + r\cos\theta\sin\Psi$

for: $x = r\cos\Psi$, $y = r\sin\Psi$

So: $x' = x\cos\theta - y\sin\theta$

$y' = x\sin\theta + y\cos\theta$

Summary

- GL History
- GL Characteristic
- A Example Using WebGL
- Shader Programing using GLSL
- Animation， Interaction using WebGL

Interaction
Working with callback

- **Callback Function/Event Listeners**
  - Programming interface(API) for event-driven input uses *"callback functions"* (回调函数）or *"event listeners"*(事件监听器)
    - ✓**Define 定义回调函数/事件监听器**
      - – ***define a callback function for each event*** **the graphics system recognizes**
    - ✓**Register 注册回调函数/事件监听器**
      - ✓***Browsers enters an event loop and responds to those events for which it has callbacks registered***
    - ✓Execute 执行回调函数/事件监听器
      - – ***The function is executed when the event occurs***
- **Ex.** use the onload window event to initiate execution of the init（）function.

  定义，注册，执行window的onload事件的回调函数init（）

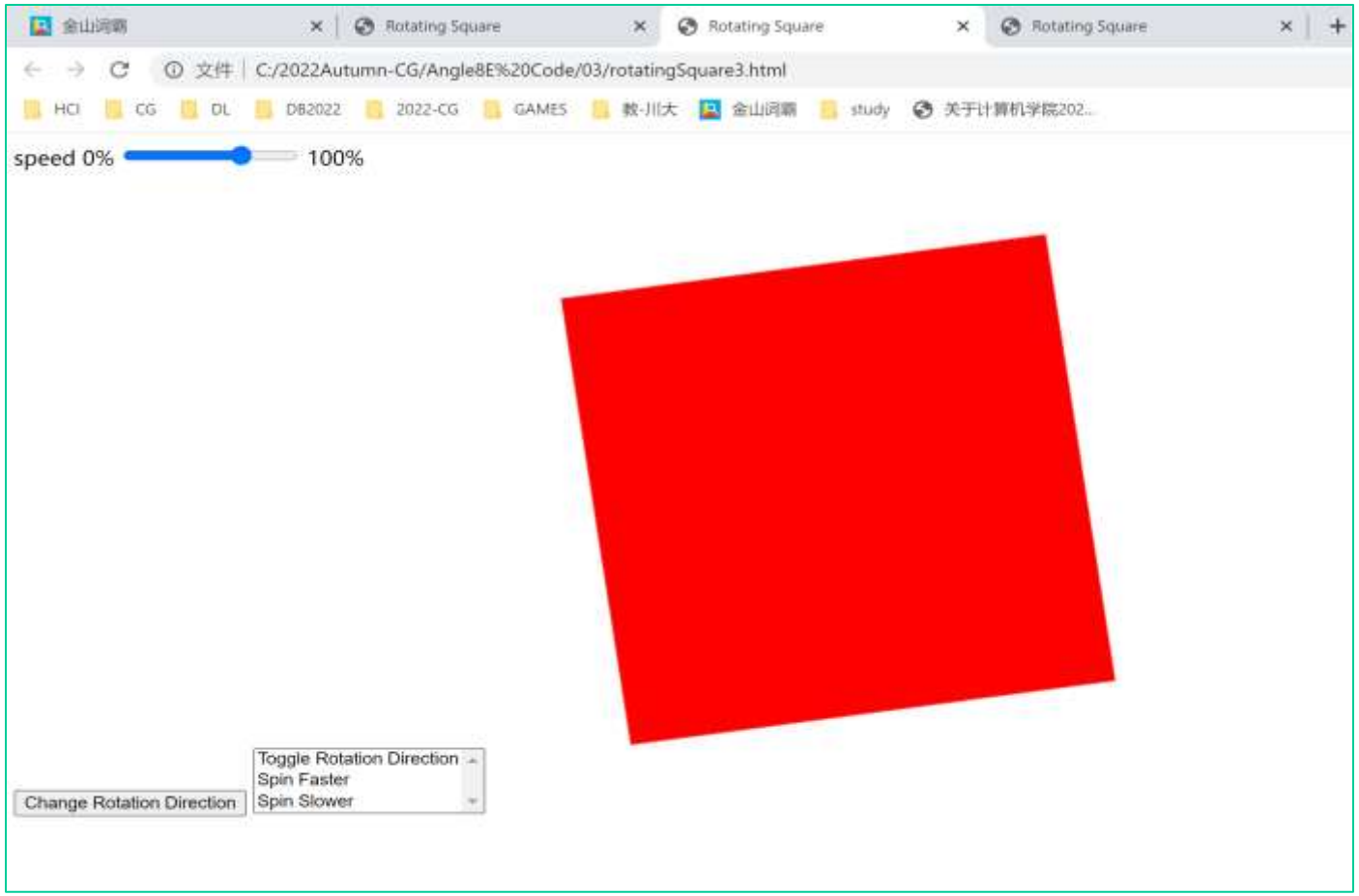
  window.onload = init(){…}； //init()相当于C中的main()函数

Interaction
Working with callback（cont.)

- 一般常用的Target（目标），Event types（事件类型）
  - Button: click,
  - Menu: click
  - Slider: change
  - Mouse: mousedown,mouseup,mousemove
  - Window: **onload**，**keydown**, **resize**，

Interaction
Working with callback（cont.）

- Example: angle8Ecode/03/rotatingSquare3



Interaction
Working with callback（cont.）

1.Adding a Button

- In the HTML file,Uses HTML button tag
  - id gives an identifier we can use in JS file
  - Text “Change Rotation Direction” displayed in button
  - Clicking on button generates a click event




```
<button id="DirectionButton">
            Change Rotation Direction
</button>
```

Interaction
Working with callback（cont.）

1.Adding a Button(cont.)

- Declare variable “direction”
  - In the render function we can use a var direction which is true or false to add or subtract a constant to the angle

```
var direction = true; // global initialization
```

```
Render(){
…
   if(direction)
        theta += 0.1;
   else
        theta -= 0.1;
   …}
```

Interaction
Working with callback（cont.）

1.Adding a Button(cont.)

- Register Button Event Listener
- **We still need to define the listener:** **no listener and the event occurs but is ignored**
- **Two forms for event listener in JS file:** **choose one**

```
var myButton = document.getElementById("DirectionButton");

myButton.addEventListener("click", function() {
    direction = !direction;
});
```

```
document.getElementById("DirectionButton").onclick =
function() {
    direction = !direction;
};
```

Interaction
Working with callback（cont.）

1.Adding a Button(cont.)

- More **onclick variants**：

```
myButton.addEventListener(“click", function(event) {
  if (event.button == 0) { direction = !direction; }
});
//Event.button=0表示是鼠标左键
```

```
myButton.addEventListener("click", function(event) {
  if (event.shiftKey == 0) { direction = !direction; }
});
//Enent.shiftkey 表示按下shift键
```

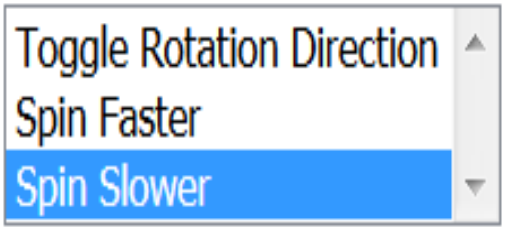
```
<button onclick="direction = !direction"></button>
```

Interaction
Working with callback（cont.）

2.Menus

- Use the HTML select element
- Each entry in the menu is an option element with an integer value returned by click event

```
<select id="mymenu" size="3">
        <option value="0">Toggle Rotation Direction</option>
        <option value="1">Spin Faster</option>
        <option value="2">Spin Slower</option>
</select>
```



Interaction
Working with callback（cont.）

- **2.Menus(cont.)**
- Add and Register Menu Listener

```
var m = document.getElementById("mymenu");
m.addEventListener("click", function() {
   switch (m.selectedIndex) {
      case 0:
         direction = !direction;
         break;
      case 1:
         delay /= 2.0;
         break;
      case 2:
         delay *= 2.0;
         break;
       }
});
```

Interaction
Working with callback（cont.)

- **3.keyboard**
  - **Using window on “keydown” Event**

```
window.addEventListener("keydown", function() {
  switch (event.keyCode) {//按键:数字键1,2, 3代替菜单选择
    case 49: // ’1’ key
      direction = !direction;
      break;
    case 50: // ’2’ key
      delay /= 2.0;
      break;
    case 51: // ’3’ key
      delay *= 2.0;
      break;
  }
});
```



Interaction
Working with callback（cont.）

- **3.keyboard(cont.)**
  - Don’t Know Unicode

```
window.onkeydown = function(event) {
  var key = String.fromCharCode(event.keyCode);
  switch (key){
    case ’1’:
      direction = !direction;
      break;
    case ’2’:
      delay /= 2.0;
      break;
    case ’3’:
      delay *= 2.0;
      break;
  }};
```

Interaction
Working with callback（cont.）

4. Slider Element

- In HTML file: Puts slider on page
  - Give it **an identifier**
  - Give it **minimum and maximum values**
  - Give it **a step size** needed to generate an event
  - Give it **an initial value**

```
<div>
speed 0 %<input id=“slider" type="range"
   min="0" max="100" step="10" value="50" />100%
</div>
```



Interaction
Working with callback（cont.)

4. Slider Element(cont.)

- Add onchange Event Listener
  - Two usages:

```
document.getElementById("slider").onchange =
function(event)
{
        delay= 100-event.target.value;
};
```

```
document.getElementById("slider").onchange =
function()
{
            delay =100- event.srcElement.value;
};
```

Interaction
CAD-like Examples

// CAD-like Examples： angleCode/03/*.*

square.html: puts a colored square at location of each mouse click

triangle.html: first three mouse clicks define first triangle of triangle strip. Each succeeding mouse clicks adds a new triangle at end of strip

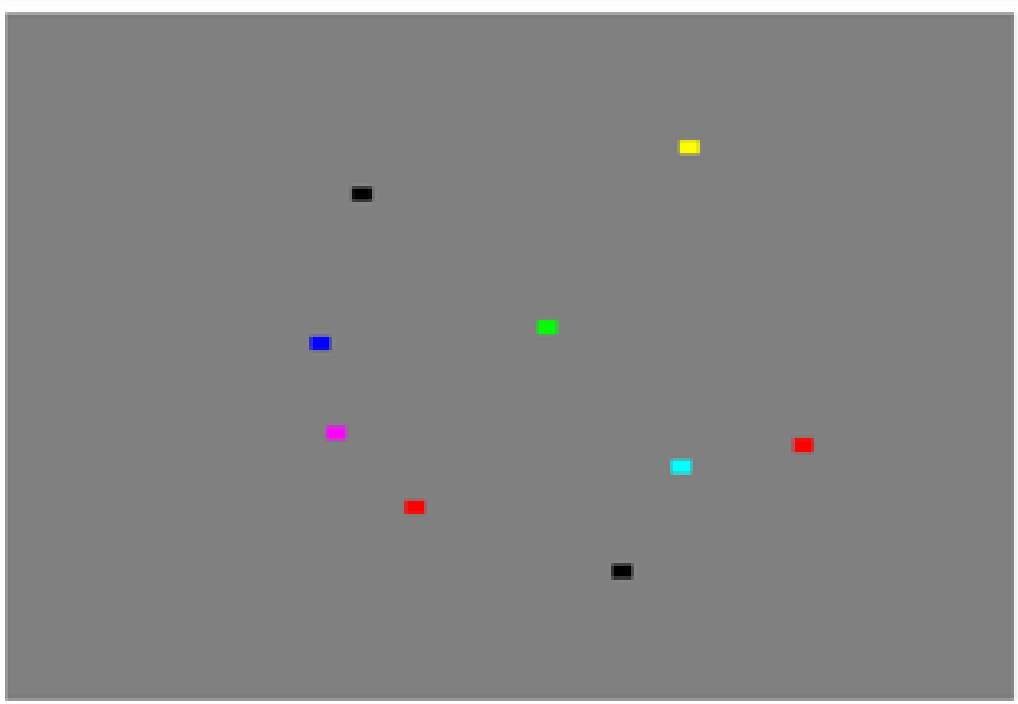
cad1.html: draw a rectangle for each two successive mouse clicks

cad2.html: draws arbitrary polygons

Interaction
CAD-like Examples(cont.）

- Position Input: use the mouse to give locations
  - Must convert from position on canvas to position in application(这里需要将屏幕桌标转换为规范化坐标）

“square.html” and “square.js”



Interaction
CAD-like Examples(cont.）

如何把鼠标击“点”的屏幕画布坐标，转换为绘制用的NDC坐标？

- Canvas specified in HTML file of size: canvas.width , canvas.height.(获取屏幕的宽度W，和高度H)
- System Returned Canvas coordinates are event.clientX and event.client（获取点的屏幕坐标Xs,Ys)

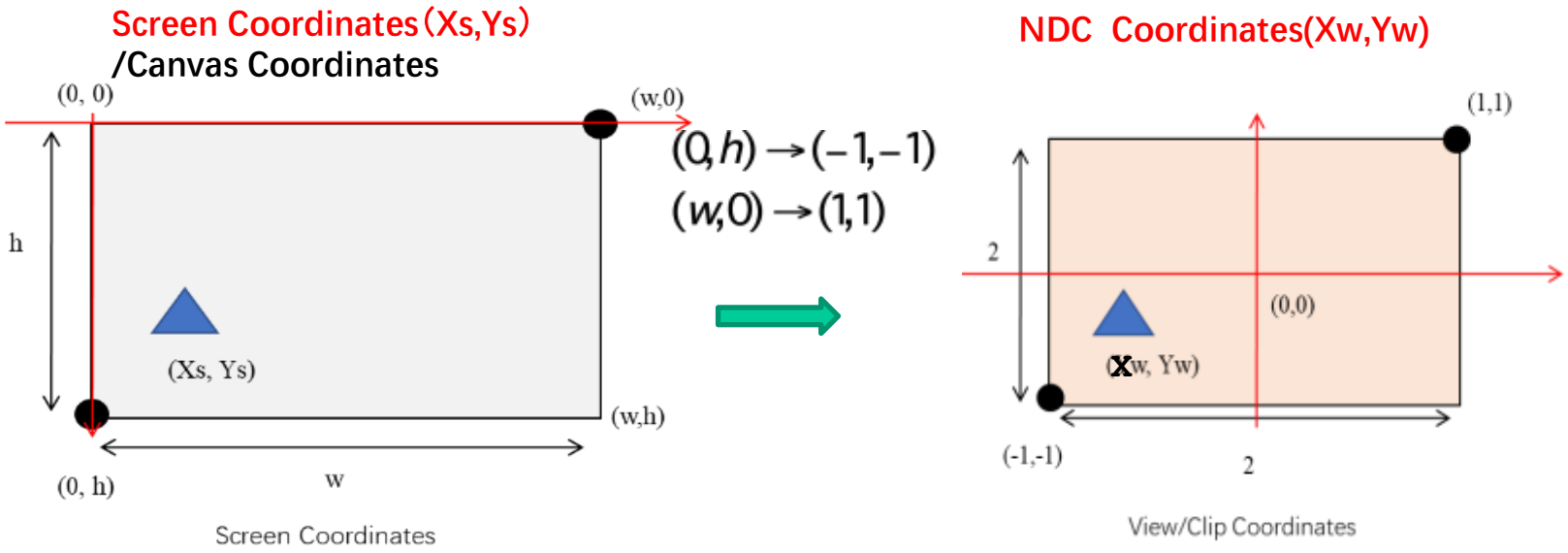
➢计算NDC下的坐标 t(x,y):

$$\frac{Xw-(-1)}{1-(-1)}=\frac{Xs-0}{w-0} \quad \text{得到 } Xw=-1+\frac{2Xs}{w}$$

$$\frac{Yw-(-1)}{1-(-1)}=\frac{Ys-h}{0-h} \quad \text{得到 } Yw=-1+\frac{2(Ys-h)}{-h}=-1+\frac{2(h-Ys)}{h}$$

```
canvas.addEventListener("mousedown", function(event){
    gl.bindBuffer( gl.ARRAY_BUFFER, vBuffer );
    var t = vec2(2*event.clientX/canvas.width-1,
       2*(canvas.height-event.clientY)/canvas.height-1);
    gl.bufferSubData(gl.ARRAY_BUFFER, 8*index, flatten(t));
```

Interaction
CAD-like Examples(cont.）



1.直接根据对应成比例进行推导：

$$\frac{Xw-Xwleft}{Xwright-Xwleft}=\frac{Xs-Xsleft}{Xsritht-Xsleft}$$

$$\frac{Yw-Ywbottom}{Ywtop-Ywbottom}=\frac{Ys-Ysbottom}{Ystop-Ysbottom}$$

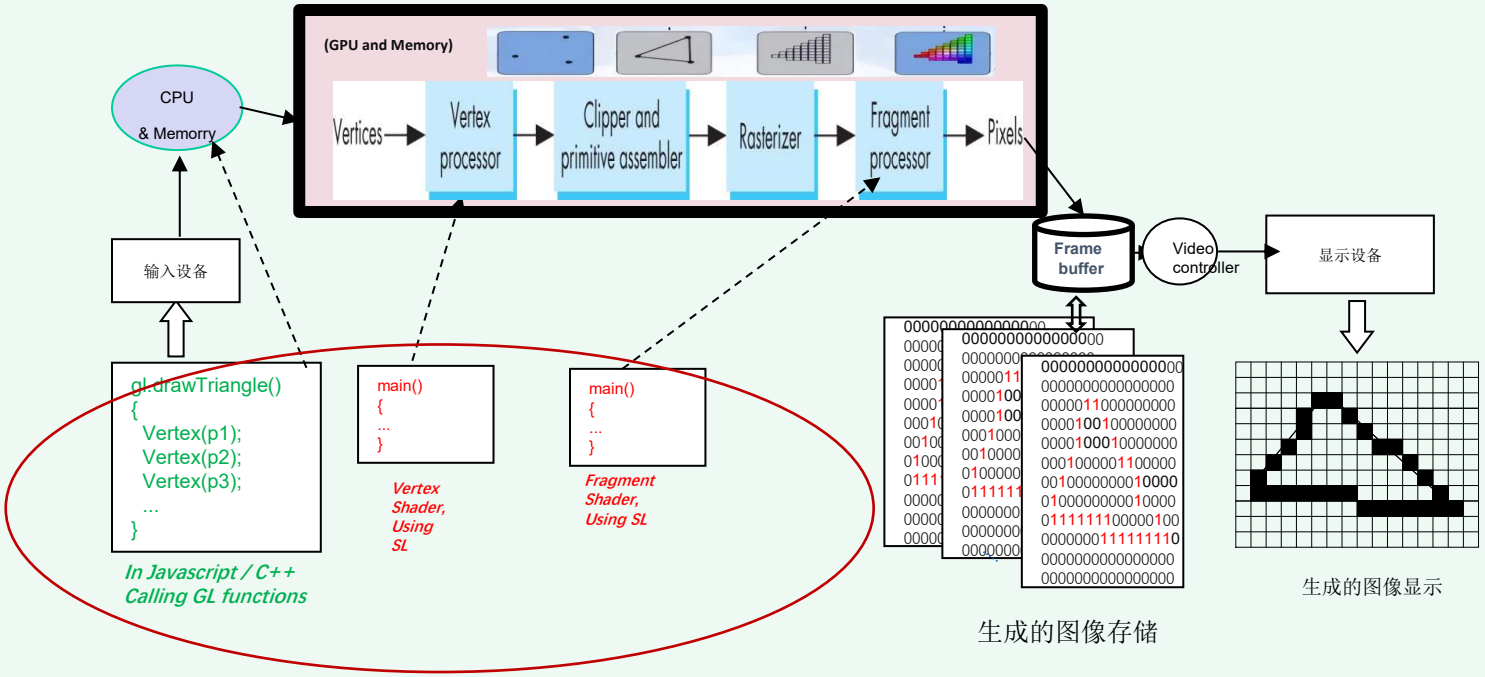
屏幕视区左上角 (xsleft, ysbottom) = (0, 0) 代入数据：

$$\frac{Xw-(-1)}{1-(-1)}=\frac{Xs-0}{w-0}$$ 得到 $Xw=-1+\frac{2Xs}{w}$

$$\frac{Yw-(-1)}{1-(-1)}=\frac{Ys-h}{0-h}$$ 得到 $Yw=-1+\frac{2(Ys-h)}{-h}=-1+\frac{2(h-Ys)}{h}$

Summary

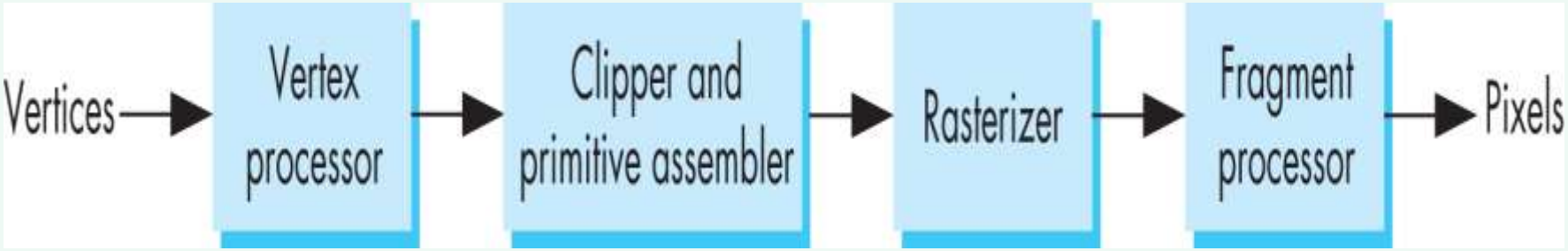
- The Programmable Rendering Pipeline 可编程渲染管线的架构
  - Performance is achieved by using GPU rather than CPU,GPU does all rendering,
  - Programmer Control GPU through shaders
  - ◆Application's job (run at CPU) is to send data to GPU
  - ◆Vertext Shader run at per vertex, fragment shader run at per fragment concurrently



Summary(cont.)

- openGL, webGL, GLSL, canvas, viewport, VBO(vertex Buffer Object)，primitive，vertext shader, fragment shader）
- Shader 中的三类变量 :attribute（in), varying(in/out) , uniform






```
//Attribute:每顶点不同的属性
in vec4 vPosition;
in vec4 vColor
```

```
//Varying: 每顶点传出值，后续将进行光栅化插值计算
out vec2 color;
```

光栅化插值

```
//Varying:每片元传入值，经过"光栅化插值"后得到的每片元的属性之值
in vec2 color;
```

```
//uniform:每顶点一致的变量
Uniform mat4 uModelMatix;
```

```
//uniform 每片元一致的变量
Uniform vec4 uLightColor;
```

Summary(cont.)

- **WebGL references:**
  - 课程QQ群文件《webGL1.0编程指南》， 参考书1例程包 Angel8ECode
  - html教程:https://www.w3school.com.cn/html/index.asp
  - js教程:https://www.w3school.com.cn/js/index.asp
  - webGL教程:
    - https://get.webgl.org/
    - https://webglfundamentals.org/webgl/lessons/zh_cn

- **GL函数查询:**

  使用方法:不需要注册登陆,右上角搜索框内输入函数名

  https://developer.mozilla.org/zh-CN/



- **OpenGL references:**

  www.opengl.org

  https://learnopengl-cn.readthedocs.io/zh/latest/